

The Hardest Explicit Construction

Oliver Korten *

May 2021

Abstract

We investigate the complexity of explicit construction problems, where the goal is to produce a particular object of size n possessing some pseudorandom property in time polynomial in n . We give overwhelming evidence that **APEPP**, defined originally by Kleinberg et al. [26], is the natural complexity class associated with explicit constructions of objects whose existence follows from the probabilistic method, by placing a variety of such construction problems in this class. We then demonstrate that a result of Jeřábek [23] on provability in Bounded Arithmetic, when reinterpreted as a reduction between search problems, shows that constructing a truth table of high circuit complexity is complete for **APEPP** under $\mathbf{P}^{\mathbf{NP}}$ reductions. This illustrates that Shannon’s classical proof of the existence of hard Boolean functions is in fact a *universal* probabilistic existence argument: derandomizing his proof implies a generic derandomization of the probabilistic method. As a corollary, we prove that $\mathbf{EXP}^{\mathbf{NP}}$ contains a language of circuit complexity $2^{n^{\Omega(1)}}$ if and only if it contains a language of circuit complexity $\frac{2^n}{2^n}$. Finally, for several of the problems shown to lie in **APEPP**, we demonstrate direct polynomial time reductions to the explicit construction of hard truth tables.

*Department of Computer Science, Columbia University. Email: oliver.korten@columbia.edu

1 Introduction

Explicit construction — the task of replacing a nonconstructive argument for the existence of a certain type of object with a deterministic algorithm that outputs one — is an important genre of computational problems, whose history is intertwined with that of some old and fundamental questions in complexity theory. The primary nonconstructive method for establishing the existence of interesting combinatorial objects is to show that a randomly chosen object has a desired property with high probability. This technique, initiated by Erdős [12] and since dubbed the “probabilistic method,” has proven immensely useful across disparate subfields of combinatorics and computer science. Indeed, the probabilistic method is currently our sole source of certainty that there exist hard Boolean functions, pseudorandom number generators, rigid matrices, and optimal randomness extractors, among a variety of other combinatorial objects.

Explicit construction problems can be phrased, in complexity terms, as *sparse* search problems¹: given the input 1^n , output some object of size n satisfying a certain property. In the interesting case, such problems are also *total*: we have a reason to believe that for all n , at least one object with this property exists. In contrast to the fundamental importance of explicit constructions, there has been surprisingly little work attempting to systematically study their complexity. This gap was pointed out previously by Santhanam [38], who studied the complexity of explicit construction problems from the following perspective: we have some property Π which is *promised* to hold for almost all strings of length n . Based on the complexity of testing the property Π , what can be said about the complexity of producing an n -bit string with property Π ? Though some interesting reductions can be shown in this framework, Santhanam notes that this approach does not seem to yield robust complexity classes with *complete* explicit construction problems.

This issue is familiar in the study of the class **TFNP**: when we have only a *promise* that a search problem is total, it is seemingly impossible to reduce it to a problem of similar complexity which has a *syntactic* guarantee of totality. This led to the study, initiated by Papadimitriou [33], of characterizing total search problems based on which combinatorial lemma guarantees the existence of a solution. In recent work of Kleinberg et al. [26], this method was used to analyse several total search problems in the polynomial hierarchy beyond **NP**. One class they define is **APEPP**, which consists of the Σ_2^P total search problems whose totality follows from the “Abundant Empty Pigeonhole Principle,” which tells us that a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ cannot be surjective (the second “P” in **APEPP** stands for “Polynomial”, referring to the polynomial-time machines defining the input f). In this paper, we show that **APEPP** is the natural *syntactic* class into which we can place a vast range of explicit construction problems where a solution is guaranteed by the probabilistic method.

Given that **APEPP** is a syntactic class, it is natural to ask whether some explicit construction problem is complete for it. As it turns out, the answer is positive: constructing a truth table of length 2^n with circuit complexity $2^{\epsilon n}$ is in fact *complete* for **APEPP** under **P^{NP}** reductions. Perhaps surprisingly, this important fact had been known for many years in the universe of Bounded Arithmetic, essentially proved in Emil Jeřábek’s PhD thesis in 2004 [23]. Here Jeřábek shows that the theorem asserting the empty pigeonhole principle is *equivalent*, in a particular theory of Bounded Arithmetic, to the theorem asserting the existence of hard Boolean functions. Although his result is phrased in terms of logical expressibility, we show that when translated to language of search problems his techniques give a **P^{NP}** reduction from any problem in **APEPP** to the problem of constructing a hard truth table. In Section 4 we give a self-contained proof of this, and generalize

¹A sparse search problem has only $n^{O(1)}$ many relevant inputs of a given length; see Section 2 for a formal definition.

the reduction to hold for arbitrary classes of circuits equipped with oracle gates. Combined with our results placing a wide range of explicit construction problems in **APEPP**, this shows that, in a concrete sense, constructing a hard truth table is a *universal explicit construction problem*. We give further credence to this claim by showing in addition that several well known explicit construction problems in **APEPP**, including the explicit construction of rigid matrices, can be directly reduced to the problem of constructing a hard truth table via polynomial time reductions (as opposed to $\mathbf{P}^{\mathbf{NP}}$ reductions).

1.1 Our Contributions

We investigate the complexity class **APEPP** introduced in [26], defined by the following complete problem **EMPTY**: given a circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ with $m > n$, find an m -bit string outside the range of C . In Section 3 we give overwhelming evidence that **APEPP** is the natural class associated with explicit constructions from the probabilistic method, by placing a wide range of well studied problems in this class. In particular, we show that the explicit construction problems associated with the following objects lie in **APEPP**:

- Truth tables of length 2^n with circuit complexity $\frac{2^n}{2n}$ (Theorem 1)
- Pseudorandom generators (Theorem 3)
- Strongly explicit two-source randomness extractors with 1 bit output for min-entropy $\log n + O(\log(1/\epsilon))$, and thus strongly explicit $O(\log n)$ -Ramsey graphs in both the bipartite and non-bipartite case (Theorem 4)
- Matrices with high rigidity over any finite field (Theorem 5)
- Strings of time-bounded Kolmogorov complexity $n - 1$ relative to any fixed polynomial time bound and any fixed Turing machine (Theorem 6)
- Communication problems outside of $\mathbf{PSPACE}^{\mathbf{CC}}$ (Theorem 13)
- Hard data structure problems in the bit-probe model (Theorem 14)

In [26], variants of Theorem 1 and Theorem 5 were shown; in that work the definition of the problems was modified so that the problems no longer had a *sparse* input, as the primary goal was to find examples of plausibly hard problems in **APEPP**, and sparse-input problems can always be solved efficiently with polynomial advice. For example, instead of the construction of hard truth tables, they studied the complexity of producing truth tables hard *relative to some oracle*, where the oracle is given as input (see Definition 6). In contrast, our focus in this work is on the sparse explicit construction variants of such problems; a major thesis of this work is that such problems, although solvable with polynomial advice, actually constitute the most interesting complexity theoretic phenomenon in the class **APEPP**.

Since the work of Impagliazzo and Wigderson [20] implies that constructing pseudorandom generators reduces to constructing hard truth tables, **APEPP** constructions of PRGs follow immediately from **APEPP** constructions of hard truth tables. However, we provide a self-contained and simple proof that PRG construction can be reduced to **EMPTY**, without requiring the more involved techniques of Nisan, Wigderson, and Impagliazzo [31][20]. Together with the result in the following section that constructing hard truth tables is complete for **APEPP** under $\mathbf{P}^{\mathbf{NP}}$ reductions, this gives an alternative and significantly simplified proof that worst-case-hard truth tables

can be used to derandomize algorithms (although it proves a weaker result, that this derandomization can be accomplished with an $\mathbf{NP}$ oracle).

In Section 4 we show that constructing a truth table of length 2^n with circuit complexity $2^{\epsilon n}$ is complete for $\mathbf{APEPP}$ under $\mathbf{P}^{\mathbf{NP}}$ reductions (for any fixed $0 < \epsilon < 1$). As discussed earlier, the core argument behind this result was proven by Jeřábek in [23], where he shows that the theorem asserting the existence of hard Boolean functions is equivalent to the theorem asserting the empty pigeonhole principle in a certain fragment of Bounded Arithmetic. We show that, when viewed through the lens of explicit construction problems, this technique yields a reduction from $\mathbf{EMPTY}$ to the explicit construction of hard truth tables. We also generalize this reduction to arbitrary oracle circuits, which allows us to prove the following more general statement: constructing a truth table which requires large $\Sigma_i^{\mathbf{P}}$ -oracle circuits is complete for $\mathbf{APEPP}_{\Sigma_i^{\mathbf{P}}}$ under $\Delta_{i+2}^{\mathbf{P}}$ reductions (the complete problem for $\mathbf{APEPP}_{\Sigma_i^{\mathbf{P}}}$ is the variant of $\mathbf{EMPTY}$ where the input circuit can have $\Sigma_i^{\mathbf{P}}$ -oracle gates). By recasting and generalizing Jeřábek's theorem in the context of explicit construction problems, we are able to derive several novel results. First and foremost, we conclude that there is a $\mathbf{P}^{\mathbf{NP}}$ construction of hard truth tables *if and only if* there is a $\mathbf{P}^{\mathbf{NP}}$ algorithm for every problem in $\mathbf{APEPP}$, and so in particular such a construction of hard truth tables would automatically imply $\mathbf{P}^{\mathbf{NP}}$ constructions for each of the well-studied problems discussed in Section 3. This tells us that constructing hard truth tables is, in a definite sense, *a universal explicit construction problem*. Since the existence of a $\mathbf{P}^{\mathbf{NP}}$ construction of hard truth tables is equivalent to the existence of a language in $\mathbf{E}^{\mathbf{NP}}$ with circuit complexity $2^{\Omega(n)}$, this completeness result actually gives an *exact algorithmic characterization* of proving $2^{\Omega(n)}$ circuit lower bounds for $\mathbf{E}^{\mathbf{NP}}$:

Theorem (Theorem 10). *There is a $\mathbf{P}^{\mathbf{NP}}$ algorithm for $\mathbf{EMPTY}$ if and only if $\mathbf{E}^{\mathbf{NP}}$ contains a language of circuit complexity $2^{\Omega(n)}$.*

As per the preceding discussion, may view this as a kind of *hardness result* for the difficulty of proving (widely-believed) exponential circuit lower bounds for $\mathbf{E}^{\mathbf{NP}}$: any such proof requires solving a universal explicit construction problem. As a corollary of this result we are also able to derive the following:

Theorem (Corollaries 2 and 3). *$\mathbf{E}^{\mathbf{NP}}$ (resp. $\mathbf{EXP}^{\mathbf{NP}}$) contains a language of circuit complexity $2^{\Omega(n)}$ (resp. $2^{n^{\Omega(1)}}$) if and only if $\mathbf{E}^{\mathbf{NP}}$ (resp. $\mathbf{EXP}^{\mathbf{NP}}$) contains a language of circuit complexity $\frac{2^n}{2^n}$.*

In both of the above theorems, we use the terminology “ $\mathbf{E}^{\mathbf{NP}}$ requires circuits of size $s(n)$ ” to indicate an almost-everywhere separation: there is a language $L \in \mathbf{E}^{\mathbf{NP}}$ so that for *all sufficiently large* n , the circuit complexity of $L \cap \{0, 1\}^n$ is at least $s(n)$.

Unpacking the proof of the above corollaries reveals an efficient algorithm to “extract hardness” from truth tables using an oracle for circuit minimization, a prospect previously considered in [5]:

Theorem (Theorem 11). *There is a polynomial time algorithm using a circuit minimization oracle (or more generally an $\mathbf{NP}$ oracle) which, given a truth table x of length M and circuit complexity s , outputs a truth table y of length $N = \Omega(\sqrt{\frac{s}{\log M}})$ and circuit complexity $\Omega(\frac{N}{\log N})$.*

We then argue, based on an observation of Williams [45], that improving this construction in its current form to extract $(\frac{s}{\log M})^{\frac{1}{2}+\epsilon}$ bits of hardness would require a breakthrough for 3SUM.

Finally, in Section 5 we consider $\mathbf{P}$ (as opposed to $\mathbf{P}^{\mathbf{NP}}$) reductions from particular explicit construction problems to the problem of constructing hard truth tables. We show that in the case of rigidity, bit probe lower bounds, and certain communication complexity lower bounds, such

reductions exist. These reductions take the following form: we show that the failure of an n -bit string x to satisfy certain pseudorandom properties implies a smaller than worst case circuit computing x . This then implies that any n -bit string of sufficiently high circuit complexity will necessarily possess a variety of pseudorandom properties, including high rigidity, high space-bounded communication complexity, and high bit-probe complexity. We also make note of an interesting dichotomy (Theorem 15), which tells us that any explicit construction problem in **APEPP** is either **APEPP**-complete under $\mathbf{P}^{\mathbf{NP}}$ reductions, or solvable in subexponential time with an **NP** oracle (for infinitely many input lengths).

Another concrete takeaway from this work is that we demonstrate, for several well-studied problems, the weakest known assumptions necessary to obtain explicit constructions of a certain type (polynomial time constructions in some cases and $\mathbf{P}^{\mathbf{NP}}$ constructions in others). Perhaps the most interesting application of this is rigidity, as the complexity of rigid matrix construction has been studied extensively in both the $\mathbf{P}$ and $\mathbf{P}^{\mathbf{NP}}$ regimes. We obtain the following conditional constructions of rigid matrices:

Theorem (Theorems 5 and 10). *If $\mathbf{E}^{\mathbf{NP}}$ contains a language of circuit complexity $2^{\Omega(n)}$, then for any prime power q there is a $\mathbf{P}^{\mathbf{NP}}$ construction of an $n \times n$ matrix over $\mathbb{F}_q$ which is $\Omega(n^2)$ -far (in hamming distance) from any rank- $\Omega(n)$ matrix.*

Theorem (Theorem 12). *If $\mathbf{E}$ contains a language of circuit complexity $\Omega(\frac{2^n}{n})$, then there is a polynomial time construction of an $n \times n$ matrix over $\mathbb{F}_2$ which is $\Omega(n^2)$ -far from any rank- $\Omega(n)$ matrix.*

In both cases, the rigidity parameters in the conclusion would be sufficient to carry out Valiant’s lower bound program [44]. The weakest hardness assumptions previously known to yield constructions with even remotely similar parameters (in either the $\mathbf{P}^{\mathbf{NP}}$ or $\mathbf{P}$ regimes) require a lower bound against *nondeterministic* circuits [29].

1.2 Related Work

A large body of work on the hardness/randomness connection, starting with that of Nisan and Wigderson [31], has exhibited the usefulness of explicit constructions of hard truth tables. The results of Impagliazzo and Wigderson [20] give, in particular, a reduction from explicit constructions of hard truth tables to explicit constructions of pseudorandom generators that fool polynomial size circuits. As noted by Santhanam [38], this immediately implies that for any “dense” property Π recognizable in $\mathbf{P}$ (dense meaning the fraction of n -bit strings holding this property is at least $1/\text{poly}(n)$), an efficient construction of a hard truth table immediately implies an efficient construction of an n -bit string with property Π . But many properties of interest such as Rigidity (or any of the other properties studied in this work) are only known to be recognizable in the larger class **NP**. Under the stronger assumption that we can construct truth tables hard for certain classes of *nondeterministic* circuits, constructions for all dense **NP** properties are known to follow as well [27] [29], so in particular $\mathbf{P}^{\mathbf{NP}}$ constructions for every problem in **APEPP** would follow. However, constructing truth tables that are hard for nondeterministic circuits appears strictly harder than constructing truth tables hard for standard circuits, and in particular does not seem to be contained in **APEPP**, so although this yields an explicit construction problem which is *hard* for **APEPP**, it does not appear to be complete. In contrast, we show here that constructing a truth table which is hard for standard circuits is both contained in and hard for **APEPP**, thus showing that a $\mathbf{P}^{\mathbf{NP}}$ construction of a hard truth table is possible *if and only if* such a construction is possible for every problem in **APEPP**.

For several of the problems we study, a long line of work has gone into improving state-of-the-art explicit constructions. We give a brief overview here of some recent work on rigid matrices and extractors. Rigidity was first introduced by Valiant [44], who showed that any matrix which is $n^{1+\epsilon}$ -far from a rank- δn matrix for some $\epsilon, \delta > 0$ cannot be computed by linear size, logarithmic depth arithmetic circuits. Since then it has been a notorious open problem to provide examples of an explicit matrix family with rigidity parameters anywhere close to this. A recent breakthrough was achieved in [1], and improved by [4], which gives $\mathbf{P}^{\mathbf{NP}}$ constructions of matrices that are $\Omega(n^2)$ -far from any rank- $2^{\log n / \Omega(\log \log n)}$ matrix, for infinitely many values of n . However, such a construction is still not sufficient to carry out Valiant’s arithmetic circuit lower bound program. A conditional result of [10] implies that $\mathbf{P}^{\mathbf{NP}}$ constructions of certain rigid matrices are possible assuming explicit constructions exist for certain hard data structure problems in the group model. In terms of polynomial time constructions, the best known construction yields a matrix which is $\frac{N^2}{\rho} \log \frac{N}{\rho}$ -far from any ρ -rigid matrix, for any parameter ρ [41][13].

The case of Ramsey graphs and extractors is slightly more complicated. There are two common definitions of the explicit construction problems corresponding to these objects. The first is often referred to as the “weakly-explicit” version, where we must output the adjacency matrix (in the case of Ramsey graphs) or truth table (in the case of extractors) in time polynomial in the size of the truth table/matrix. The second version, referred to as the “strongly-explicit” version, is to output a succinct circuit which computes the adjacency relation or extractor function. Clearly the strongly-explicit case is harder, but in both cases, there is a significant gap between what is achievable by explicit methods and what can be proven possible by the probabilistic method. We will focus on the strongly explicit case in this work, and in the case of extractors we will focus on *two-source* extractors with one bit of output, which remains the most challenging current frontier [6]. The state of the art constructions for both two-source extractors and Ramsey graphs are due to Li [28]. He demonstrates a two-source extractor for constant error and min-entropy $O(\log n)$, and hence an n -vertex graph which is $\log^{O(1)} n$ -Ramsey. Our results show that strongly explicit extractors for min-entropy $\log n + O(\log(1/\epsilon))$, and thus n -vertex $O(\log n)$ -Ramsey graphs, can be constructed in **APEPP**. In both cases these parameters are known to be the best possible. For a comprehensive, somewhat up-to-date survey on extractors see [6] (this survey predates the above-stated construction in [28] but covers much of the work which lead up to it).

Another line of work in the area of explicit constructions investigates the possibility of *pseudodeterministic* constructions of certain objects. Here, the construction algorithm is allowed to use randomness, but must output the *same object* on most computation paths. Originally introduced in [14], this paradigm was recently applied in a pair of works [8, 32] to the construction of large prime numbers, where a polynomial time pseudodeterministic construction which works for infinitely many input lengths is given. This algorithm works more generally for the problem of constructing elements of a language L which is (1) recognizable in polynomial time, and (2) satisfies $|L \cap \{0, 1\}^n| \geq 2^n/n^{O(1)}$ for all n .

1.3 Proof Sketch of Main Theorem

We give here an informal overview of the proof that **EMPTY** can be solved in polynomial time given access to a hard truth table and an $\mathbf{NP}$ oracle. At the core of this proof is a familiar construction in the theory of computing which dates back to the 1980’s, namely the pseudorandom function generator of Goldreich, Goldwasser, and Micali [15]. Note that in the following, we will refer to the “circuit complexity of an n -bit string x ” to mean the size of the smallest circuit computing x_i given i in binary; this is well-defined even when n is not a power of 2, as we shall formalize in Section 3².

²See Definition 4

Consider the special case of EMPTY where our input is a circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ which exactly doubles its input size. For a moment let us forget our primary goal of finding a $2n$ -bit string outside C 's range, and instead consider C as a cryptographic pseudorandom generator which we are attempting to break. Since C is a function which extends its input size by a positive number of bits, it is indeed of the same syntactic form as a cryptographic PRG, so this viewpoint is well-defined.

In [15], Goldreich, Goldwasser and Micali give a procedure³ which, for any fixed $0 < \epsilon < 1$, takes C and produces in polynomial time a new circuit $C^* : \{0, 1\}^n \rightarrow \{0, 1\}^m$ for some $m = \text{poly}(n)$, which satisfies the following two properties:

- (1) Every string in the range of C^* has circuit complexity at most m^ϵ .
- (2) Given a statistical test breaking C^* , we can construct a statistical test of similar complexity breaking C .

The construction of C^* is in fact quite simple: for an appropriate choice of k , we recursively apply C to an n -bit input for k iterations as follows: first apply C to an n -bit string to get two n -bit strings, then apply it again to each of those to get 4, and continue k times until we obtain 2^k many n -bit strings.

A key observation made by Razborov and Rudich [36] is that condition (1) automatically implies a *particular statistical test which breaks C^** , namely the test which accepts precisely those m -bit strings with circuit complexity exceeding m^ϵ . But by property (2), C^* inherits the security of C , which is an arbitrary candidate PRG. This means that determining if an m -bit string has circuits of size m^ϵ is in fact a *universal test for randomness*, capable of simultaneously breaking all pseudorandom generators.

Recall now our original goal for C , which was to find a $2n$ -bit string outside its range. Property (1) of C^* implies that an explicit construction of a length- m truth table of circuit complexity m^ϵ would immediately yield an explicit m -bit string outside the range of C^* . In Section 4, we show that C^* obeys the following third property:

- (3) Given a string outside the range of C^* , we can find a string outside the range of C using a polynomial number of calls to an **NP** oracle.

The analogue of statement (3) in the context of Bounded Arithmetic was first shown by Jeřábek [23], and a quite similar argument appears even earlier in the work of Paris, Wilkie, and Woods [34]. Combining properties (1) and (3), we get the desired result: any m -bit string of complexity m^ϵ must lie outside the range of C^* , so using such a string together with an **NP** oracle we can solve our original instance of EMPTY.

To summarize, the construction C^* of Goldreich, Goldwasser and Micali shows that the property of requiring large circuits is a *universal pseudorandom property of strings* in two concrete senses:

- (a) (Original analysis of [15] and [36]) A test determining whether a string requires large circuits can be efficiently bootstrapped into a test distinguishing any pseudorandom distribution from the uniform distribution.
- (b) (This work together with [23]) An explicit example of a string requiring large circuits can be used (together with an **NP**-oracle) to generate an explicit example of a string outside the range of any efficiently computable map $C : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ (in fact any $C : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ as shown in Section 4), and in particular can be used to construct explicit examples of strings possessing each of the fundamental pseudorandom properties examined in Section 3.

³[15] and [36] apply the construction described here in a different parameter regime, so our statement of the result differs slightly from its original presentation. The version described here has been noted subsequently in the literature on MCSP, see for example [39].

2 Definitions

Following [26], we define the set of total functions in Σ_2^P , denoted $\mathbf{TF}\Sigma_2^P$, as follows:

Definition 1. A relation $R(x, y)$ is in $\mathbf{TF}\Sigma_2^P$ if there exists a polynomial $p(n)$ such that the following conditions hold:

1. For every x , there exists a y such that $|y| \leq p(|x|)$ and $R(x, y)$ holds
2. There is a polynomial time Turing machine M such that for all x, y ,
 $R(x, y) \iff \forall z \in \{0, 1\}^{p(|x|)} M(x, y, z)$ accepts

The search problem associated with such a relation is: “given x , find some y such that $R(x, y)$ holds.” For the majority of this paper, we will be concerned primarily with *sparse* $\mathbf{TF}\Sigma_2^P$ search problems, where the only relevant part of the input is its length. We can thus define the following “sparse” subclass of $\mathbf{TF}\Sigma_2^P$:

Definition 2. A relation $R(x, y)$ is in $\mathbf{STF}\Sigma_2^P$ if $R \in \mathbf{TF}\Sigma_2^P$ and for any x_1, x_2 such that $|x_1| = |x_2|$, we have that for all y , $R(x_1, y) \iff R(x_2, y)$.

Since the length of x fully determines the set of solutions, the relevant search problem here is: “given 1^n , find some y such that $R(1^n, y)$ holds.” All explicit construction problems considered in Section 3 will be in $\mathbf{STF}\Sigma_2^P$ (with the exception of **COMPLEXITY** which we briefly mention as it was studied previously in [26]).

We now define the search problem **EMPTY**, which will be the primary subject of this work:

Definition 3. **EMPTY** is the following search problem: given a Boolean circuit C with n input wires and m output wires where $m > n$, find an m -bit string outside the range of C .

Following the original publication of this work, [37] renamed the the problem “**EMPTY**” as “**RANGE AVOIDANCE**” or simply “**AVOID**,” which has become standard in all succeeding work on the topic.

This problem is total due to the basic lemma, referred to in [26] as the “Empty Pigeonhole Principle” and in the field of Bounded Arithmetic as the “Dual Pigeonhole Principle [23],” which tells us that a map from a smaller set onto a larger one cannot be surjective. Since verifying a solution y consists of determining that for all x , $C(x) \neq y$, we have:

Observation 1. **EMPTY** $\in \mathbf{TF}\Sigma_2^P$

Since for any instance of **EMPTY** the number of output bits m is at least $n + 1$, a random m -bit string will be a solution with probability at least $\frac{1}{2}$. Since verifying a solution can be accomplished with one call to an **NP** oracle, this implies the following inclusion:

Observation 2. **EMPTY** $\in \mathbf{FZPP}^{\mathbf{NP}}$

As mentioned in the introduction, this fact tells us that sufficiently strong pseudorandom generators capable of fooling *nondeterministic* circuits such as those in [27] would suffice to derandomize the above inclusion and yield a $\mathbf{P}^{\mathbf{NP}}$ algorithm for **EMPTY**. In Section 4, we will show that this derandomization can be accomplished under a significantly weaker assumption, using a reduction of a very different form than in the hardness-based pseudorandom generators of [31], [20], and [27].

We can now define the class **APEPP**, which is simply the class of search problems polynomial-time reducible to **EMPTY**. This class was originally defined in [26], and is an abbreviation for

“Abundant Polynomial Empty Pigeonhole Principle.” The term “Abundant” was used to distinguish this from the larger class **PEPP** also studied in [26]. The complete problem for **PEPP** is to find a string outside the range of a map $C : \{0, 1\}^n \setminus \{0^n\} \rightarrow \{0, 1\}^n$, which appears significantly more difficult (it is at least as hard as **NP** [26]). The distinction between **APEPP** and **PEPP** also appears in the Bounded Arithmetic literature, where the principle corresponding to **APEPP** is referred to as the “Dual *weak* Pigeonhole Principle,” while the principle corresponding to **PEPP** is referred to simply as the “Dual Pigeonhole Principle.” We will be concerned only with the abundant/weak principle in this work. It should be noted that we employ a slight change of notation from [26] for the sake of simplicity: we use **EMPTY** to refer to the search problem associated with the *weak* pigeonhole principle, while in [26] **EMPTY** refers to the search problem associated with the full pigeonhole principle.

Infinitely-often vs. almost-everywhere circuit lower bounds: As a final point of clarification, whenever we make the statement “ L requires circuits of size $s(n)$ ” for some language L and size bound s , we mean that circuits of size $s(n)$ are required to compute L on length n inputs for *all but finitely many* n . This is in contrast to the statement “ $L \notin \mathbf{SIZE}(s(n))$,” which means the circuit size lower bound holds for infinitely many input lengths. All circuit lower bounds referred to in this work will be of the first kind.

3 Explicit Constructions in APEPP

In this section, we show that a variety of well-studied explicit construction problems can be reduced in polynomial time to **EMPTY**. Each proof follows roughly the following format: there is some property of interest Π , and our goal is to construct an n -bit string which has this property. Whenever an n -bit string x fails to have property Π , it indicates that x is somehow more “structured” than a random n -bit string, and this structure allows us to specify x using fewer than n bits. We then actualize this argument in the form of an efficiently computable map $C : \{0, 1\}^k \rightarrow \{0, 1\}^n$ with $k < n$, such that any string not having property Π is in the range of C . This immediately implies that any n -bit string outside the range of C must hold property Π , and thus any solution to the instance of **EMPTY** defined by C will be a solution to our explicit construction problem. For many of the proofs, we will only show that the reduction is valid for n sufficiently large; clearly this is sufficient, since explicit constructions can be done by brute force for fixed input lengths.

A useful coding lemma: In the proofs to come, it will be helpful to utilize succinct and efficiently computable encodings of low-weight strings (the “weight” of binary string is the number of 1 bits it contains). We start with the following folklore result, reproduced in [16], which gives an optimal encoding of weight- k n -bit strings:

Lemma 1. [16] *For any $k \leq n$, there exists a map $\Phi : \{0, 1\}^{\lceil \log \binom{n}{k} \rceil} \rightarrow \{0, 1\}^n$ computable in $\text{poly}(n)$ time such that every n -bit string of weight k is in the range of Φ .*

We typically use this in combination with the following standard bounds which hold for all $n \in \mathbb{N}$ and $\epsilon \in (0, 1)$:

1. $\log \binom{n}{(\frac{1}{2} - \epsilon)n} \leq n - \epsilon^2 n$
2. $\log \binom{n}{\epsilon n} \leq H(\epsilon)n$

Here $H(\cdot)$ denotes the binary entropy function. If we want to encode a string of weight *at most* $(\frac{1}{2} - \epsilon)n$ (resp. ϵn) it will be sufficient for our purposes to take a union bound over the possible values $k \leq (\frac{1}{2} - \epsilon)n$ of the weight. Since $\log \binom{n}{k}$ is increasing in k up to $k = \lfloor \frac{n}{2} \rfloor$ this will incur at most an additive $\lceil \log n \rceil$ bit increase in the encoding length over the above bounds, e.g. it will allow us to encode a string of weight at most $(\frac{1}{2} - \epsilon)n$ using $\leq \lceil n - \epsilon^2 n \rceil + \lceil \log n \rceil$ bits.

3.1 Hard Truth Tables

Definition 4. Given a string x of length N , we say that x is computed by a circuit of size s if there is a Boolean circuit C of fan-in 2 over the basis $\langle \wedge, \vee, \neg \rangle$ with $\lceil \log N \rceil$ inputs and s gates, such that $C(i) = x_i$ for all $1 \leq i \leq |x|$. If N is not a power of 2, we put no restriction on the value of $C(i)$ for $i > |x|$.

Definition 5. **HARD TRUTH TABLE** is the following search problem: given 1^N , output a Boolean string x of length N such that x is not computed by any circuit of size at most $\frac{N}{2 \log N}$.

In the typical case where $N = 2^n$ for some n , this is equivalent to finding a truth table for an n -input Boolean function requiring circuits of size $\frac{2^n}{2n}$, which is within a $2 + o(1)$ factor of the maximum possible circuit complexity for any n -input Boolean function.

Theorem 1. **HARD TRUTH TABLE** reduces in polynomial time to **EMPTY**.

Proof. This proof follows Shannon’s classical argument for the existence of functions of high circuit complexity [40]. We construct an instance of **EMPTY** in the form of a circuit Φ which maps an encoding of a circuit to its corresponding truth table. Φ interprets its input as a circuit on $\lceil \log N \rceil$ bits (using an encoding of circuits to be described below), evaluates it on every possible input to generate a $2^{\lceil \log N \rceil}$ bit truth table, and then truncates this truth table to be of length exactly N .

We now describe the encoding of circuits used by Φ , which will guarantee that any N -bit string x with a circuit of size $\frac{N}{2 \log N}$ is in the range of Φ . Given a circuit of size $s \geq \lceil \log N \rceil$ on $\lceil \log N \rceil$ inputs computing x , for each of its s gates we can use 2 bits to encode whether it is an $\wedge, \vee$, or $\neg$ gate, and an additional $2 \log(s + \lceil \log N \rceil) \leq 2 \log s + O(1)$ bits to specify its inputs. We can then use an additional $\log s$ bits to specify which gate is the terminal output gate. Overall this requires $2s \log s + O(s)$ bits. It is clear that from such an encoding, Φ can efficiently decode the represented circuit and evaluate it on all possible input values. For $s \leq \frac{N}{2 \log N}$, we have:

$$2s \log s + O(s) \leq \frac{N}{\log N} \log \left(\frac{N}{2 \log N} \right) + O \left(\frac{N}{\log N} \right) = N - \Omega \left(\frac{N \log \log N}{\log N} \right) + O \left(\frac{N}{\log N} \right)$$

which is strictly less than N for N sufficiently large. So Φ is indeed a valid instance of **EMPTY**, and any string outside the range of Φ is a solution to **HARD TRUTH TABLE**. It is also clear from the above description that Φ can be constructed in $\text{poly}(N)$ time. □

Following the original publication of this work, it was shown in [7] that the above argument could be sharpened using a more involved circuit encoding procedure, so that truth tables of essentially optimal hardness $\frac{N}{\log N}$ could be constructed via reduction to **EMPTY**.

A related problem was studied in [26], referred to there as “COMPLEXITY.” Rather than an explicit construction problem with sparse input, in this problem you are given as input one truth table x of length N , and asked to produce another truth table y such that y requires large circuits, even with access to x -oracle gates. More formally:

Definition 6. The problem COMPLEXITY is defined as follows: given an N -bit string x , find another N -bit string y such that y requires x -oracle circuits of size $\frac{N}{\log^2 N}$.

Note that the “ x oracle” is not the typical definition of an oracle gate that can solve arbitrarily sized instances of a fixed language, but rather an oracle for a fixed Boolean function on $\log N$ variables. In [26] the following is shown:

Theorem 2. COMPLEXITY reduces in polynomial time to EMPTY.

It should be noted that COMPLEXITY bears resemblance to a problem studied by Ilango [18], termed the “Minimum Oracle Circuit Size Problem,” or “MOCSP.” In this problem, the input consists of two truth tables x and y and a size parameter s , and the goal is to determine if y has x -oracle circuits of size at most s . Ilango demonstrates that MOCSP is **NP**-complete under randomized reductions.

3.2 Pseudorandom Generators

Definition 7. We will say that a sequence $R = (x_1, \dots, x_m)$ of n -bit strings is a pseudorandom generator if, for all n -input circuits of size n :

$$|Pr_{x \sim R}[C(x) = 1] - Pr_{y \sim \{0,1\}^n}[C(y) = 1]| \leq 1/n$$

Standard applications of the probabilistic method show that such pseudorandom generators exist of size polynomial in n . Thus we can define the following total search problem:

Definition 8. PRG is the following search problem: given 1^n , output a pseudorandom generator $R = (x_1, \dots, x_m)$, $x_i \in \{0,1\}^n$.

A polynomial time algorithm for PRG would suffice to derandomize **BPP** [31]. We now show how to formalize the argument for the totality of PRG using the empty pigeonhole principle. In particular, we show that a PRG of size n^6 can be constructed in **APEPP**.

As noted in the introduction, the results of Impagliazzo and Wigderson [20] imply that PRG reduces directly to HARD TRUTH TABLE, so a reduction of PRG to EMPTY follows from Theorem 1. However, we provide here a much simpler direct proof that PRG reduces to EMPTY, relying only on Yao’s next bit predictor lemma, and neither the nearly disjoint subsets construction of Nisan and Wigderson [31] nor the rather involved worst-case to average-case reductions of Impagliazzo and Wigderson [20]. Together with our completeness result in Section 4, this gives an alternative, self-contained proof that worst-case-hard truth tables can be used to construct pseudorandom generators (although it yields a weaker result, as our derandomization will require an **NP** oracle).

The basic idea of the proof is as follows: if R fails to be a pseudorandom generator for circuits of size n , then applying Yao’s lemma there exists a next bit predictor circuit of size $O(n)$ which can predict the i^{th} bit of most strings in R from the previous $i - 1$ bits. This will allow us to compress R by omitting the i^{th} bits of all the strings in R and replacing them with the predictor D and some advice bits which will correct the errors made by D .

Theorem 3. PRG reduces in polynomial time to EMPTY

Proof. We start by constructing the following circuit Φ . Φ interprets its input as representing a list $R^- = (x_1, \dots, x_{n^6})$ of n^6 strings of length $n - 1$, a circuit D of size cn (for a fixed universal constant c to be defined later) with $n - 1$ input bits and one output bit, a $\log n$ -bit index $i \in [n]$, and a string S encoding an n^6 -bit string with weight at most $\frac{n^6}{2} - n^4$. Given these, Φ feeds each

$n - 1$ -bit string $x_j \in R^-$ through D , then inserts the output as an extra bit at the i^{th} position of x_j to obtain an n -bit string x_j^* . In the final step, Φ decodes the n^6 -bit string represented by S , and flips the i^{th} bit of x_j^* if and only if the j^{th} position of S has a 1, to obtain x_j' . Φ then outputs $R = (x_1', \dots, x_{n^6}')$.

It is clear that Φ can be implemented as a circuit of size polynomial in n , and further that Φ can be constructed in polynomial time given 1^n . We now show there are fewer inputs than outputs. Note that the input size is equal to the number of bits needed to specify R^-, D, i, S , which is $n^7 - n^6 + \tilde{O}(n) + \log n + \text{bits}(S)$, where $\text{bits}(S)$ is the number of bits needed to specify S . Since S is an n^6 -bit string with weight at most $\frac{n^6}{2} - n^4 = n^6(\frac{1}{2} - \frac{1}{n^2})$, we can apply Lemma 1 with $\epsilon = \frac{1}{n^2}$ to give an encoding for S using $n^6(1 - \frac{1}{n^4}) + \log(n^6) = n^6 - n^2 + 6 \log n$ bits. Thus the overall number of bits of input is at most $n^7 - n^6 + \tilde{O}(n) + \log n + n^6 - n^2 + 6 \log n = n^7 + \tilde{O}(n) + 7 \log n - n^2$ which is strictly less than the n^7 bits of output (for sufficiently large n). So this is indeed a polynomial time reduction to a valid instance of EMPTY. It remains to show that any string outside the range of Φ is a pseudorandom generator.

Let R be a sequence of n -bit strings of size n^6 which fails to be a pseudorandom generator; we aim to show that R lies in the range of Φ . By definition, there exists a circuit C of size n such that:

$$|Pr_{x \sim R}[C(x) = 1] - Pr_{y \sim \{0,1\}^n}[C(y) = 1]| > 1/n$$

For $x \in \{0,1\}^n$ and $i \in [n]$, let x^{-i} be the $n - 1$ -bit string obtained by deleting the i^{th} bit of x , and let x^i denote the i^{th} bit of x . By Yao's next bit predictor lemma [46] [43], the previous inequality implies the existence of an index $i \in [n]$ and a circuit $D : \{0,1\}^{n-1} \rightarrow \{0,1\}$ of size cn for some fixed universal constant c , such that

$$Pr_{x \sim R}[D(x^{-i}) = x^i] > \frac{1}{2} + \frac{1}{n^2}$$

Since R has size n^6 , this implies that D correctly guesses the i^{th} bit of $x \in R$ from the other $n - 1$ bits for at least $n^6(\frac{1}{2} + \frac{1}{n^2}) = \frac{n^6}{2} + n^4$ of the elements of R . So if we let S be the n^6 -bit string with a 1 at the indices where D guesses the wrong value of x^i , we see that the weight of S is at most $\frac{n^6}{2} - n^4$ as required. Taking $R^- = (x_1^{-i}, \dots, x_{n^6}^{-i})$, we have that from R^-, D, i, S we can efficiently deduce R .

So overall, we have established that for any R which is not a pseudorandom generator, there exists some R^-, D, i, S such that our circuit Φ outputs R on input R^-, D, i, S . Thus, any string outside the range of Φ is a pseudorandom generator. $\square$

3.3 Strongly Explicit Randomness Extractors and Ramsey Graphs

A (k, ϵ) two-source extractor with one bit of output is a function $f : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ such that for any pair of distributions X, Y on $\{0,1\}^n$ of min-entropy at least k , the value of $f(xy)$ for a random x, y in the product distribution of X, Y is ϵ -close to an unbiased coin flip. By a well-known simplification of [9], to show that a function f is a (k, ϵ) extractor, it suffices to show that it satisfies the above condition for every pair of "flat" k -sources X, Y , which are uniform distributions over subsets of $\{0,1\}^n$ of size 2^k . We will thus use the following definition of two-source extractors to define our explicit construction problem:

Definition 9. We say that a function $f : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ is a (k, ϵ) extractor if the following holds: for any two sets $X, Y \subseteq \{0,1\}^n$ of size 2^k , $|Pr_{x \sim X, y \sim Y}[f(xy) = 1] - \frac{1}{2}| \leq \epsilon$.

Definition 10. For any pair of functions $k, \epsilon : \mathbb{N} \rightarrow \mathbb{N}$, (k, ϵ) -EXTRACTOR is the following search problem: given 1^n , output a circuit C with $2n$ inputs such that the function $f_C : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ defined by C is a $(k(n), \epsilon(n))$ extractor.

The above problem definition does not expressly constrain the size of C , though for a construction to be “explicit” in any useful sense (efficiently computable as a function of n), C would have to have size polynomial in n . The following reduction placing extractor construction in **APEPP** will immediately imply that we can construct a $(\log n + O(1), \epsilon)$ extractor of circuit size approximately n^3 in **APEPP** for any fixed ϵ . The fact that good extractors exist with small circuit complexity, and in fact small depth-3 $\mathbf{AC}^0[\oplus]$ circuit complexity, was first shown by Razborov [35]. His approach was to construct a k -wise independent distribution of functions computable by small circuits, an approach which we will follow. To turn the argument into a reduction to EMPTY we will instead use the more standard family of k -wise independent random functions arising from low degree univariate polynomials over a large finite field (in our case we do not care so precisely about the structure of the circuit computing our function).

Theorem 4. For any efficiently computable $\epsilon(n)$ satisfying $\frac{1}{n^c} < \epsilon(n) < \frac{1}{2}$ for a constant c and sufficiently large n , $(\log n + 2 \log(1/\epsilon(n)) + 3, \epsilon(n))$ -EXTRACTOR reduces in polynomial time to EMPTY.

Proof. Let $\epsilon = \epsilon(n)$, and let $d = \lceil \frac{4}{\epsilon^2} \rceil$. We will set up an instance of EMPTY with at most $2d^2n^3 + 2dn^2 - \epsilon^2d^2n^2 + 2 \log(2dn) + 1$ inputs and exactly $2d^2n^3$ outputs which has the following property: Let A be any $2d^2n^3$ -bit string outside the range of this circuit, viewed as an ordered list of d^2n^2 elements of $\mathbb{F}_{2^{2n}}$ denoted $\alpha_1, \dots, \alpha_{d^2n^2}$, and consider the function $f : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$ defined by

$$f(x) = \sum_{i=1}^{d^2n^2} \alpha_i x^{i-1}$$

Then the function $g : \{0, 1\}^{2n} \rightarrow \{0, 1\}$ defined by

$$g(x) = \text{the last bit of } f(x)$$

is a $(\log dn, \epsilon)$ extractor. Since $\log dn = \log n + \log d \leq \log n + \log(4/\epsilon^2 + 1) \leq \log n + 2 \log(1/\epsilon) + 3$, this would give the required result.

Let $\alpha_1, \dots, \alpha_{d^2n^2}$ be any sequence of coefficients in $\mathbb{F}_{2^{2n}}$ such that the above function g corresponding to the α_i is not a $(\log dn, \epsilon)$ extractor. So there exist two sets of n -bit strings X, Y , each of size $2^{\log dn} = dn$, and some $b \in \{0, 1\}$ such that $\Pr_{x \sim X, y \sim Y}[g(xy) = b] > \frac{1}{2} + \epsilon$. Let $R = \{xy \mid x \in X, y \in Y\} \subseteq \{0, 1\}^{2n}$. We have $|R| = |X||Y| = d^2n^2$. Let $r_1, \dots, r_{d^2n^2}$ denote the lexicographical enumeration of R . By assumption, the last bit of $f(r_i)$ is equal to b for at least a $\frac{1}{2} + \epsilon$ fraction of indices i . So then, if we let β_i be the $2n - 1$ -bit prefix of $f(r_i)$, we can deduce the value of $f(r_i)$ from β_i and b for at least $d^2n^2(\frac{1}{2} + \epsilon)$ values of i . Thus, there is some $d^2n^2(\frac{1}{2} - \epsilon)$ -weight d^2n^2 -bit string S , such that from b, S , and the β_i 's, we can deduce $f(r_i)$ for all i . Now, once we are able to deduce $f(x)$ for each of the d^2n^2 distinct values of x in R , since f is a degree $d^2n^2 - 1$ polynomial, we can uniquely and efficiently determine the coefficients α_i of f using Gaussian elimination on the corresponding $d^2n^2 \times d^2n^2$ Vandermonde matrix.

So we have shown that for any set of α_i 's which does not define a $(\log dn, \epsilon)$ extractor, there exists X, Y, b, S , and β_i 's, from which we can efficiently deduce the α_i 's. It is clear that we can encode X, Y using $2dn^2$ bits, b using 1 bit, and the β_i 's using $d^2n^2(2n - 1) = 2d^2n^3 - d^2n^2$ bits. Since S is a $d^2n^2(\frac{1}{2} - \epsilon)$ -weight d^2n^2 -bit string, by Lemma 1, we can encode S using at most

$d^2n^2(1 - \epsilon^2) + \log(d^2n^2) = d^2n^2 - \epsilon^2d^2n^2 + \log(d^2n^2)$ bits. So in total the number of input bits is at most:

$$2dn^2 + 1 + d^2n^2 - \epsilon^2d^2n^2 + \log(d^2n^2) + 2d^2n^3 - d^2n^2 = 2d^2n^3 + (2d - \epsilon^2d^2)n^2 + 2\log(dn) + 1$$

Since we chose $\frac{4}{\epsilon^2} + 1 \geq d \geq \frac{4}{\epsilon^2}$, we have that $(2d - \epsilon^2d^2) < -1$ for $\epsilon \leq \frac{1}{2}$, so the number of input bits is at most $2d^2n^3 - n^2 + 2\log(dn) + 1$. Since we assumed $\epsilon \geq \frac{1}{n^c}$ for a constant c , we have $\epsilon \geq \frac{1}{2n^{2/5}}$ for sufficiently large n , and so $n^2 > 2\log(dn) + 1$, which is strictly less than the number of output bits $2d^2n^3$.

So this implies that the circuit Φ mapping X, Y, b, S and the β_i 's to a corresponding set of α_i 's is a valid instance of EMPTY, and that any solution to this instance is a set of coefficients defining a $(\log n + 2\log(1/\epsilon) + 3, \epsilon)$ extractor. From the coefficients α_i we can easily construct an efficient circuit computing the function g , thus solving the problem $(\log n + 2\log(1/\epsilon) + 3, \epsilon)$ -EXTRACTOR. It is also clear from the construction that this reduction can be carried out in $\text{poly}(n, \frac{1}{\epsilon})$ time, so $\text{poly}(n)$ time under our assumption that $\epsilon(n) \geq \frac{1}{n^c}$ for a constant c . $\square$

For the typical parameter regime where ϵ is an arbitrarily small constant, this gives a two source extractor for min-entropy $\log n + O(1)$.

Corollary 1. *Explicit construction of strongly explicit Ramsey graphs (n -vertex graphs containing no clique or independent set of size $c \log n$ for some constant c), in both the bipartite and non-bipartite case, reduces to EMPTY.*

Proof. As noted in [3], any two-source extractor in the above sense (with ϵ fixed to any constant less than one half) is automatically a bipartite Ramsey graph, and from a strongly explicit bipartite Ramsey graph we can construct a strongly explicit non-bipartite Ramsey graph efficiently. $\square$

3.4 Rigid Matrices

Definition 11. [44] *We say that $n \times n$ matrix M over $\mathbb{F}_q$ is (r, s) rigid if for any matrix $S \in \mathbb{F}_q^{n \times n}$ with at most s non-zero entries, $M + S$ has rank greater than r .*

Definition 12. *For any $q : \mathbb{N} \rightarrow \mathbb{N}$ such that $q(n)$ is a prime power for all n , (ϵ, q) -RIGID is the following search problem: given 1^n , output an $n \times n$ matrix M over $\mathbb{F}_{q(n)}$ which is $(\epsilon n, \epsilon n^2)$ rigid.*

Theorem 5. *For any constant $\epsilon > 0$ so that $H(\epsilon) + 3\epsilon < 1$, and any efficiently computable $q(n)$ satisfying the above, (ϵ, q) -RIGID reduces in polynomial time to EMPTY.*

In particular $\epsilon = \frac{1}{8}$ suffices.

Proof. Let M be any $n \times n$ matrix over $\mathbb{F}_q$ which is not (r, s) rigid. So there exists an $n \times r$ matrix L , an $r \times n$ matrix R , and an $n \times n$ matrix S with at most s non-zero entries, such that $M = LR + S$. It is clear that from the descriptions of L, R, S we can efficiently compute M .

L and R can each be described explicitly using $nr \log q$ bits. For S , we encode it by specifying an n^2 -bit string T of weight s denoting the entries of S which are nonzero, together with an $s \log q$ -bit string C giving the correct values of the nonzero entries. Using this scheme we may assume s is exactly equal to ϵn^2 since if it is smaller we can add some arbitrary indices and put their unchanged values in C . Applying the encoding scheme in Lemma 1 for T , overall the number of bits in this

encoding is at most $\log \binom{n^2}{s} + (2nr + s) \log q$. Setting $r = \epsilon n$ and $s = \epsilon n^2$ this is at most:

$$\begin{aligned} \log \binom{n^2}{\epsilon n^2} + (3\epsilon n^2) \log q &\leq \\ H(\epsilon)n^2 + (3\epsilon n^2) \log q &\leq \\ (H(\epsilon) + 3\epsilon)n^2 \log q &\end{aligned}$$

By our choice of ϵ this is bounded above by $(1 - \Omega(1))n^2 \log q$ bits, whereas an arbitrary matrix in $\mathbb{F}_q^{n \times n}$ requires $n^2 \log q$ bits. $\square$

3.5 Strings of High Time-Bounded Kolmogorov Complexity

Definition 13. Let U be any fixed Turing machine, and let $t : \mathbb{N} \rightarrow \mathbb{N}$ be a time bound. For a string x , $K_U^t(x)$ denotes the length of the smallest string y such that U outputs x on input y in $t(|x|)$ steps.

Definition 14. For a Turing machine U and time bound t , we define the following explicit construction problem K_U^t -RANDOM: given 1^n , output an n bit string x such that $K_U^t(x) \geq n - 1$

Theorem 6. For any fixed Turing machine U and fixed polynomial time bound t , K_U^t -RANDOM reduces in polynomial time to EMPTY.

Proof. We construct a circuit Φ with $n - 1$ input bits and n output bits as follows. If the input is of the form 0^*1y (0^* denotes an arbitrary length string of zeros), Φ simulates U on y for $t(n)$ steps and outputs the result, padded/truncated to length n if it is not already. Otherwise, Φ outputs 0^n . It is clear that Φ can be produced in polynomial time given 1^n for any fixed U, t . Further, for any n -bit string x with $K_U^t(x) \leq n - 2$, there is some y of length $k \leq n - 2$ such that U outputs x on input y in $t(n)$ steps, and so Φ will output x on input $0^{n-k-2}1y$. So any string x outside the range of Φ must have $K_U^t(x) \geq n - 1$. $\square$

3.6 Other Problems

In Section 5, we introduce two more explicit construction problems and show that each of these, in addition to a variant of the rigidity problem, can be reduced directly to HARD TRUTH TABLE in polynomial time. This also implies that both problems are contained in **APEPP**. We will postpone a formal definition of each of these new problems until Section 5, but give an informal statement here for completeness:

Explicit communication problems outside $\text{PSPACE}^{\text{CC}}$: An explicit $2^n \times 2^n$ communication matrix which cannot be solved by any $o(n)$ -space protocol can be constructed in **APEPP**.

Explicit data structure problems with high bit-probe complexity: An explicit data structure problem with nearly maximum complexity in the bit-probe model can be constructed in **APEPP**.

4 Constructing Hard Truth Tables is Complete for APEPP

In this section we show that constructing a hard truth table is complete for **APEPP** under $\mathbf{P}^{\text{NP}}$ reductions. As mentioned before, the core of this theorem was originally proven by Jeřábek [23],

and the main construction underlying the reduction dates back further to the work of Goldreich, Goldwasser, and Micali [15]. Jeřábek’s result is phrased in the language of proof complexity, stating that the theorem asserting the existence of hard Boolean functions is *equivalent* to the empty pigeonhole principle in a particular theory of Bounded Arithmetic. We demonstrate below that when translated to the language of search problems and explicit constructions, his proof yields a $\mathbf{P}^{\mathbf{NP}}$ reduction from EMPTY to the problem of constructing a hard truth table. We in fact prove a more general statement here which holds for arbitrary circuit classes equipped with oracle gates. For a very broad set of circuit classes $\mathcal{C}$, we prove that given a truth table which is hard for $\mathcal{C}$ -circuits, we can find an empty pigeonhole of any $\mathcal{C}$ -circuit *using polynomially many calls to an oracle for inverting $\mathcal{C}$ -circuits* (by inverting we mean finding the preimage of a given string, or reporting that none exist). In order to make this precise, we first define the type of generalized circuit classes we will consider:

Definition 15. A circuit class is defined by a basis $\mathcal{C}$, which is simply a (possibly infinite) set of Boolean-valued Boolean functions. A $\mathcal{C}$ -circuit is then defined as a circuit composed entirely of gates computing functions in the basis $\mathcal{C}$. For a language L , we will refer to the “basis L ” to mean the basis $\{L_n \mid n \in \mathbb{N}\} \cup \{\wedge, \vee, \neg\}$, where L_n is the n -bit Boolean function deciding L on length n inputs. For a complexity class $\mathbf{C}$ with a complete language L , we will refer to “the basis $\mathbf{C}$ ” to mean the basis corresponding to L . The size of a $\mathcal{C}$ -circuit is the number of wires.

We say that a basis $\mathcal{C}$ is “sufficiently strong” if there exist $\mathcal{C}$ -circuits computing the two-input $\wedge, \vee$ functions and the one-input $\neg$ function.

For any basis $\mathcal{C}$, a “ $\mathcal{C}$ -inverter oracle” is an oracle which, given a $\mathcal{C}$ -circuit C and some string y , determines whether there exists an x such that $C(x) = y$, and produces such an x if it exists. A $\mathcal{C}$ -inverter reduction is a polynomial time reduction that uses a $\mathcal{C}$ -inverter oracle.

Note that in the special case where $\mathcal{C} = \{\wedge, \vee, \neg\}$, a $\mathcal{C}$ -inverter reduction is equivalent to a $\mathbf{P}^{\mathbf{NP}}$ reduction (since inverting a circuit over the standard Boolean basis is $\mathbf{NP}$ -complete).

Definition 16. For any basis $\mathcal{C}$, the class of search problems $\mathbf{APEPP}^{\mathcal{C}}$ is defined by the following complete problem $\text{EMPTY}^{\mathcal{C}}$: given a $\mathcal{C}$ -circuit with more output wires than input wires, find a Boolean string whose length is equal to the number of output wires but which is not in the range of this circuit. For any strictly increasing function $f : \mathbb{N} \rightarrow \mathbb{N}$, we define the problem $\text{EMPTY}_{f(n)}^{\mathcal{C}}$, which is the special case of $\text{EMPTY}^{\mathcal{C}}$ where the circuit is required to have $f(n)$ output wires, where n is the number of input wires.

We start with the following technical lemma, which allows us to restrict our attention to circuits with exactly twice as many outputs as inputs.

Lemma 2. For any basis $\mathcal{C}$, $\text{EMPTY}_{2n}^{\mathcal{C}}$ is complete for $\mathbf{APEPP}^{\mathcal{C}}$ under $\mathcal{C}$ -inverter reductions.

Proof. It is straightforward to see that $\text{EMPTY}_{n+1}^{\mathcal{C}}$ is complete for $\mathbf{APEPP}^{\mathcal{C}}$: given a $\mathcal{C}$ -circuit C with n inputs and more than n outputs, we can simply delete all the output bits except for the first $n + 1$ to obtain an instance of $\text{EMPTY}_{n+1}^{\mathcal{C}}$. Any $n + 1$ -bit string outside the range of this smaller circuit can then be padded arbitrarily to the output size of the original circuit, and this padded string must also be outside the range of C .

We now reduce $\text{EMPTY}_{n+1}^{\mathcal{C}}$ to $\text{EMPTY}_{2n}^{\mathcal{C}}$. Let C be some $\mathcal{C}$ -circuit with n inputs and $n + 1$ outputs. For a positive integer i , we define $C^i : \{0, 1\}^n \rightarrow \{0, 1\}^{n+i}$ inductively as follows. For the base case, C^1 is simply defined as C . For $i > 1$, we first compute C^{i-1} on the input x to obtain a string x' of length $n + i - 1$, and then compute C on the first n bits of x' and concatenate the

output with the remaining bits of x' . Since C replaces the first n bits with $n + 1$ bits, if C^{i-1} has output length $n + i - 1$ then C^i has output length $n + i$.

We now claim that for any positive i , if we are given an $n + i$ -bit string outside the range of C^i , we can find an n -bit string outside the range of C in $\text{poly}(i|C|)$ time using a $\mathcal{C}$ -inverter. In particular, we prove by induction on i that this can be accomplished with i inverter calls. For $i = 1$ this is trivially the case, since $C^1 = C$. Now say $i > 1$. Let $y \in \{0, 1\}^{n+i}$ be outside the range of C^i for some $x \in \{0, 1\}^{n+1}$, $z \in \{0, 1\}^{i-1}$. We first use the inverter oracle to determine if x has a preimage under C ; if it does not then we have found an empty pigeonhole for C and are done. Otherwise, we use the inverter to find a preimage $x' \in \{0, 1\}^n$ for x under C . So then $x'z$ must be outside the range of C^{i-1} , since if it were not then $xz = y$ would have a preimage under C^i , contradicting our initial assumption. So by induction we can use $i - 1$ inverter calls to find an empty pigeonhole for C given $x'z$, completing the proof of the inductive case.

Now, setting $i = n$, we get a reduction from $\text{EMPTY}_{n+1}^{\mathcal{C}}$ to $\text{EMPTY}_{2n}^{\mathcal{C}}$ as claimed. $\square$

We now define the hard truth table construction problem that will be used in our reduction:

Definition 17. Let $\epsilon\text{-HARD}^{\mathcal{C}}$ denote the following search problem: given 1^N , output a string x of length N such that x cannot be computed by $\mathcal{C}$ -circuits of size N^ϵ .

In the case where $\mathcal{C} = \{\wedge, \vee, \neg\}$, we drop the subscript and refer to this problem simply as $\epsilon\text{-HARD}$. For $N = 2^n$, a solution to $\epsilon\text{-HARD}$ on input 1^N is a truth table of a function on n variables requiring $2^{\epsilon n}$ -sized circuits, the same object used to build the Impagliazzo-Wigderson generator [20].

Theorem 7. Let $\mathcal{C}$ be any sufficiently strong basis and $\epsilon > 0$ be a constant such that $\epsilon\text{-HARD}^{\mathcal{C}}$ is total for sufficiently large input lengths. Then $\text{EMPTY}^{\mathcal{C}}$ reduces in polynomial time to $\epsilon\text{-HARD}^{\mathcal{C}}$ under $\mathcal{C}$ -inverter reductions.

Proof. By Lemma 2 we know that $\text{EMPTY}^{\mathcal{C}}$ reduces to $\text{EMPTY}_{2n}^{\mathcal{C}}$ under $\mathcal{C}$ -inverter reductions. Now, let C be an instance of $\text{EMPTY}_{2n}^{\mathcal{C}}$, and let $k = 2\lceil \log |C| \rceil \lceil \frac{1}{\epsilon} \rceil$. Consider the following map $C^* : \{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$, defined informally as follows: given a string $x \in \{0, 1\}^n$, apply C once to get two n -bit strings, then apply C to both of those n -bit strings to get four, and continue k times until we have 2^k many n -bit strings, or equivalently a $2^k n$ -bit string. This process is illustrated in Figure 1 below:

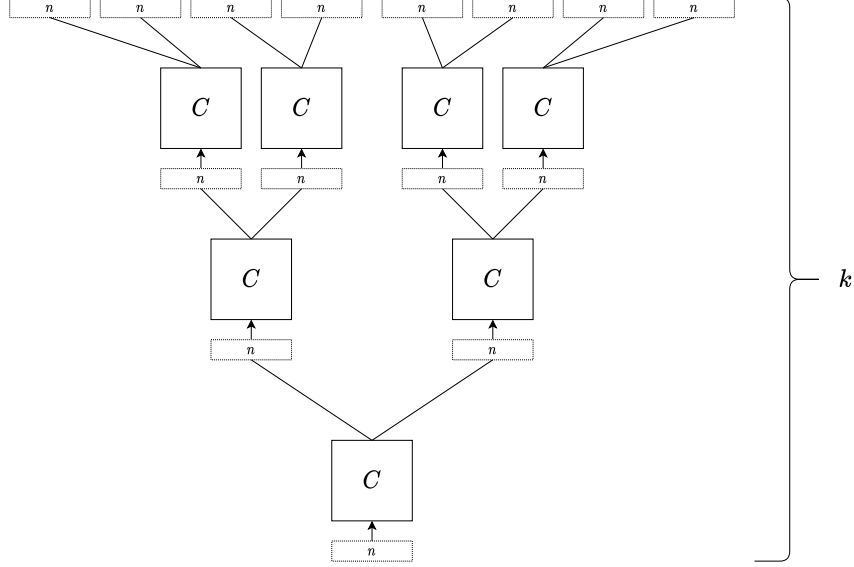


Figure 1: Extending a map $C : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ to a map $C^* : \{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$. Dotted boxes indicate the number of bits along a wire.

To define this function more formally, first we define the following maps $L, R : \{0, 1\}^{2n} \rightarrow \{0, 1\}^n$, where L takes a $2n$ -bit string and outputs the first n bits, and R takes a $2n$ -bit string and outputs the last n bits. Given a nonempty sequence $\sigma_1, \dots, \sigma_t \in \{L, R\}^*$, let $C^\sigma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be the function $\sigma_t \circ C \circ \dots \circ \sigma_1 \circ C$. Now, given a binary string, we can associate it with such a sequence by associating 0 with L and 1 with R , and so we will abuse notation and write C^x for a binary string x as shorthand for C^σ where σ is the sequence of L, R associated with the binary string x . We are now ready to formally define our function C^* . As C^* is a map $\{0, 1\}^n \rightarrow \{0, 1\}^{2^k n}$, we can think of the output as being clumped into 2^k blocks, each containing n bits. Given this terminology, C^* behaves as follows: on input x , the i^{th} block of the output of C^* will be $C^{\|i\|}(x)$, where $\|i\|$ denotes the standard representation of i as a k -bit binary string.

From here, the proof proceeds in two steps. First, we show that, by setting $m = n2^k = \text{poly}(|C|)$, any solution to $\epsilon\text{-HARD}^C$ on input 1^m will be a string that is not in the range of C^* . Second, we will show that given a string outside the range of C^* , we can find a string outside the range of C using only a polynomial number of calls to a C -inverter.

To carry out the first of these steps, we will show that any string in the range of C^* , when interpreted as a truth table of length $m = n2^k$ on $\lceil \log n \rceil + k$ variables, can be computed by a circuit of size $O(|C|k)$. Since, by construction of k , we have that $m \geq |C|^{\frac{2}{\epsilon}}$, a solution to $\epsilon\text{-HARD}$ on input 1^m will be a truth table of length m not computable by a circuit of size $m^\epsilon \geq |C|^2$, and thus a circuit of size $O(|C|k) = O(|C| \log |C|)$ would be a contradiction for all input lengths greater than some absolute constant. We construct such a circuit for any string in the range of C^* as follows: let y be a $2^k n$ -bit string such that for some $x \in \{0, 1\}^n$, $C^*(x) = y$. The circuit computing y will have x written as advice/constants, and will feed x through k copies of the circuit C in series. We will split the $\lceil \log n \rceil + k$ input variables into a block of k variables we call i , and a block of $\lceil \log n \rceil$ variables we call j . We then use i to determine whether to apply L or R to the output of one of the copies of C before feeding it into the next, to get some resulting string x^i , and then we use j to index into the j^{th} position of x^i , to get $y_{i,j}$. A diagram of this circuit is shown in Figure 2:

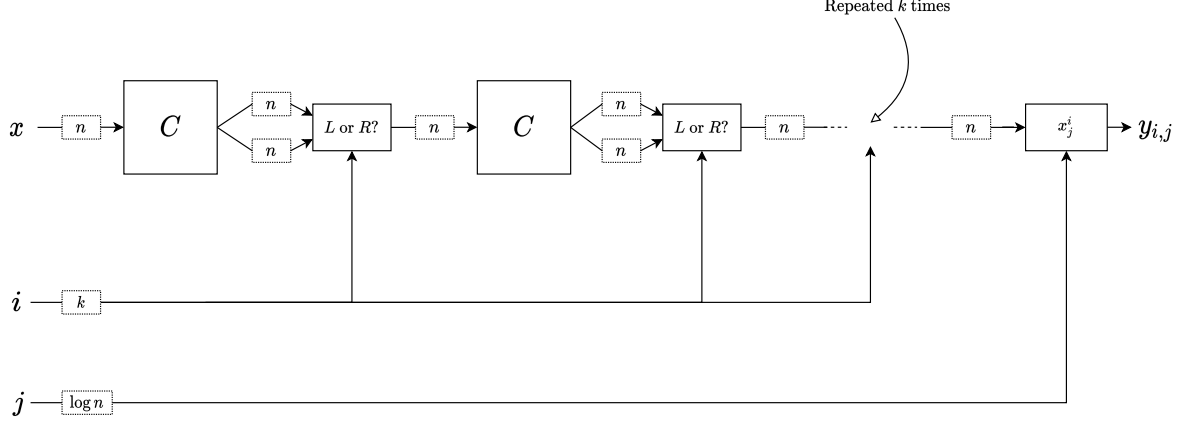


Figure 2: A succinct circuit whose truth table is y , for any y in the range of C^* . Dotted boxes indicate the number of bits along a wire. Note that although x is shown as an input in this diagram, for any given y we fix a preimage x as constants/advice, and so the only true inputs to this circuit are i, j .

To see that this circuit has size $O(|C|k)$, note that the subcircuits computing either L or R depending on a bit of i can be computed easily with $O(n)$ gates over the basis $\{\wedge, \vee, \neg\}$ (this is essentially a multiplexer), and also that the final subcircuit indexing into an n -bit string can be computed with $O(n)$ $\{\wedge, \vee, \neg\}$ gates as well; since we assumed C is sufficiently strong, both of these subcircuits can therefore be computed $O(n)$ -size C -circuits. Since $|C| \geq n$, and this circuit contains only k copies of C and the aforementioned subcircuits, plus the constants describing the string x of length n , this circuit has $O(|C|k)$ size as claimed.

Thus, we now know that any solution to ϵ -HARD C on input 1^m will not be in the range of C^* , and by assumption ϵ -HARD C is total for sufficiently large input lengths so such a solution exists. It remains only to show that we can use a string outside the range of C^* , together with a C -inverter oracle, to find a string outside the range of C . We proceed exactly as in the proof of Lemma 2. Let y be any string outside the range of C^* . Refer to Figure 1 which gives a diagram of a circuit computing C^* ; at a layer $i \in [k]$ of this circuit, we have 2^i blocks of n bits feeding into 2^i copies of C , and these copies of C then output 2^{i+1} blocks of n bits at the next layer. So working back from the output layer k , we can test if any consecutive $2n$ -bit block of y is outside of the range of C . If none of them are, then we find a preimage for all blocks, interpret this as the output of the previous layer, and continue our search from there. We follow this process all the way back to the input layer or until we find an empty pigeonhole of C . If we never find an empty pigeonhole of C , then this process will terminate at the input layer with a string x such that $C^*(x) = y$, which is impossible by assumption, so at some point we must indeed find a string outside the range of C . Checking whether a particular string is an empty pigeonhole, or finding a preimage if it's not, can be accomplished with one call to a C -inverter oracle by definition. We perform this test at most $2^k = \text{poly}(|C|)$ times (once for every copy of C in the diagram in Figure 1), so overall this process can be accomplished in polynomial time using a C -inverter oracle. $\square$

We now examine the implications of this theorem for particular circuit classes of interest.

Theorem 8. For any $0 < \epsilon < \frac{1}{2}$, ϵ -HARD $^{\Sigma_i^P}$ is complete for $\mathbf{APEPP}^{\Sigma_i^P}$ under $\mathbf{P}^{\Sigma_{i+1}^P}$ reductions.

Here we use “ $\mathbf{P}^{\Sigma_{i+1}^P}$ reduction” to denote a polynomial time reduction using a Σ_{i+1}^P oracle.

Proof. Containment of $\epsilon\text{-HARD}^{\Sigma_i^P}$ in $\mathbf{APEPP}^{\Sigma_i^P}$ follows directly from the proof of Theorem 1 with minimal adjustments; we must add the assumption $\epsilon < \frac{1}{2}$ to account for the unbounded fan-in of oracle gates in our counting argument. So it remains only to show that $\epsilon\text{-HARD}^{\Sigma_i^P}$ is hard for this class as well. Since the above reduction uses a polynomial number of calls to the $\mathcal{C}$ -inverter, it suffices to show that we can implement a Σ_i^P -circuit inverter with polynomially many calls to a Σ_{i+1}^P oracle. This can be accomplished by a standard application of binary search, observing that for any Σ_i^P -oracle circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ and any $x \in \{0, 1\}^n, y \in \{0, 1\}^m$, the predicate $\exists x' < x \ C(x') = y$ lies in Σ_{i+1}^P . $\square$

In the absence of any oracle gates, we have the following:

Theorem 9. *For any $0 < \epsilon < 1$, $\epsilon\text{-HARD}$ is complete for $\mathbf{APEPP}$ under $\mathbf{P}^{\mathbf{NP}}$ reductions.*

4.1 Implications of Completeness

This result gives an *exact algorithmic characterization* of the possibility of proving $2^{\Omega(n)}$ Σ_i^P -circuit lower bounds for $\mathbf{E}^{\Sigma_{i+1}^P}$:

Theorem 10. *There exists a language in $\mathbf{E}^{\Sigma_{i+1}^P}$ with Σ_i^P -circuit complexity $2^{\Omega(n)}$ if and only if there is a $\mathbf{P}^{\Sigma_{i+1}^P}$ algorithm for $\text{EMPTY}^{\Sigma_i^P}$.*

Proof. Say there is a language L in $\mathbf{E}^{\Sigma_{i+1}^P}$ with Σ_i^P -circuit complexity $2^{\Omega(n)}$. So there exists an $\epsilon > 0$ such that for all but finitely many n , L cannot be computed on length n inputs with Σ_i^P -circuits of size less than $2^{\epsilon n}$. So then we have a polynomial time algorithm for $\frac{\epsilon}{2}\text{-HARD}^{\Sigma_i^P}$ as follows: given 1^n , output the truth table of L over $\lfloor \log n \rfloor$ -bit inputs. Since $L \in \mathbf{E}^{\Sigma_{i+1}^P}$, this can be done in $2^{\lfloor \log n \rfloor} 2^{O(\log n)} = \text{poly}(n)$ time with a Σ_{i+1}^P oracle. This truth table will have length $\frac{n}{2} \leq 2^{\lfloor \log n \rfloor} \leq n$. We then pad this truth table with 0's at the end to be of length n . If there was a circuit of size $n^{\epsilon/2}$ for this n -bit truth table on $\lfloor \log n \rfloor$ bits, then on the first $\lfloor \log n \rfloor$ bits of input this computes the truth table for L on $\lfloor \log n \rfloor$ -bit inputs. Since $\frac{n}{2} \leq 2^{\lfloor \log n \rfloor}$, this would imply a circuit of size $2^{\epsilon \lfloor \log n \rfloor}$ to compute L on $\lfloor \log n \rfloor$ -bit inputs, contradicting the hardness assumption. Thus, there exists a $\mathbf{P}^{\Sigma_{i+1}^P}$ algorithm for $\frac{\epsilon}{2}\text{-HARD}^{\Sigma_i^P}$ for some $\epsilon > 0$, and so by Theorem 9, there also exists a $\mathbf{P}^{\Sigma_{i+1}^P}$ algorithm for $\text{EMPTY}^{\Sigma_i^P}$.

Alternatively, say there is a $\mathbf{P}^{\Sigma_{i+1}^P}$ algorithm for $\text{EMPTY}^{\Sigma_i^P}$. So in particular there is a $\mathbf{P}^{\Sigma_{i+1}^P}$ algorithm for $\epsilon\text{-HARD}^{\Sigma_i^P}$ for any fixed $\epsilon < \frac{1}{2}$. Consider the language L decided by the following $\mathbf{E}^{\Sigma_{i+1}^P}$ machine: given an n -bit input, we use our Δ_{i+2}^P algorithm for $\epsilon\text{-HARD}^{\Sigma_i^P}$ on input 1^{2^n} to generate a truth table, then look up the n -bit input in this truth table to determine whether to accept or reject. By definition this language must have Σ_i^P -circuit complexity $2^{\Omega(n)}$, and this machine will run in time $\text{poly}(2^n) = 2^{O(n)}$ with a Σ_{i+1}^P oracle. $\square$

In the most interesting case, we conclude that a $2^{\Omega(n)}$ circuit lower bound for $\mathbf{E}^{\mathbf{NP}}$ holds *if and only if* there is a $\mathbf{P}^{\mathbf{NP}}$ algorithm for EMPTY . Together with the results in Section 3, this gives newfound insight into the difficulty of proving exponential circuit lower bounds for the class $\mathbf{E}^{\mathbf{NP}}$: proving such a lower bound requires solving a *universal explicit construction problem*, and would immediately imply $\mathbf{P}^{\mathbf{NP}}$ constructions for a vast range of combinatorial objects which we currently have no means of constructing without a Σ_2^P oracle. Theorem 10 also allows us to derive the following interesting fact about the circuit complexity of $\mathbf{E}^{\mathbf{NP}}$:

Corollary 2 (Worst-Case to Worst-Case Hardness Amplification in $\mathbf{E}^{\mathbf{NP}}$). *If there is a language in $\mathbf{E}^{\mathbf{NP}}$ of circuit complexity $2^{\Omega(n)}$, then there is a language in $\mathbf{E}^{\mathbf{NP}}$ requiring circuits of size $\frac{2^n}{2n}$.*

Proof. By Theorem 10, if there is a language in $\mathbf{E}^{\mathbf{NP}}$ of circuit complexity $2^{\Omega(n)}$, then there is a $\mathbf{P}^{\mathbf{NP}}$ algorithm for **EMPTY**. By Theorem 1, this implies a $\mathbf{P}^{\mathbf{NP}}$ algorithm for **HARD TRUTH TABLE**, and thus a $\mathbf{P}^{\mathbf{NP}}$ construction of a truth table of length N with hardness $\frac{N}{2 \log N}$. This in turn implies the existence of a language in $\mathbf{E}^{\mathbf{NP}}$ of circuit complexity $\frac{2^n}{2n}$. $\square$

Tweaking the proof of Theorem 7 slightly we also obtain the following:

Corollary 3 (Worst-Case to Worst-Case Hardness Amplification in $\mathbf{EXP}^{\mathbf{NP}}$). *If there is a language in $\mathbf{EXP}^{\mathbf{NP}}$ of circuit complexity $2^{n^{\Omega(1)}}$, then there is a language in $\mathbf{EXP}^{\mathbf{NP}}$ requiring circuits of size $\frac{2^n}{2n}$.*

Proof. The proof follows that of the previous corollary, with the following modification to the reduction in Theorem 7: we start with the assumption that for some $\epsilon > 0$ we are able to construct N -bit truth tables with hardness $2^{\log^\epsilon N}$ in time quasipolynomial in N using an $\mathbf{NP}$ oracle, and then apply the same reduction setting $k = 2 \log^{\lceil \frac{1}{\epsilon} \rceil} |C|$. $\square$

We thus obtain a rather unexpected “collapse” theorem for the circuit complexity of $\mathbf{EXP}^{\mathbf{NP}}$: if $\mathbf{EXP}^{\mathbf{NP}}$ has circuits of size $\frac{2^n}{2n}$ infinitely often, then this class in fact has circuits of size 2^{n^ϵ} infinitely often for every $\epsilon > 0$.

We can refine this slightly as follows.

Definition 18. *MCSP, defined originally in [25], is the following decision problem: given a truth table x and a size parameter s , determine whether x has a circuit of size at most s . Let **SEARCH-MCSP** denote the search variant of this problem, where we are given a truth table x and must output a circuit computing x of minimum size.*

For the hardness amplification procedures in Corollaries 2 and 3, we can in fact replace the $\mathbf{NP}$ oracle with an oracle for **SEARCH-MCSP**, which is non-trivial since **SEARCH-MCSP** is not known to be $\mathbf{NP}$ -hard.

Corollary 4. *If there is a language in $\mathbf{E}^{\mathbf{Search-MCSP}}$ (resp. $\mathbf{EXP}^{\mathbf{Search-MCSP}}$) of circuit complexity $2^{\Omega(n)}$ (resp. $2^{n^{\Omega(1)}}$), then there is a language in $\mathbf{E}^{\mathbf{Search-MCSP}}$ (resp. $\mathbf{EXP}^{\mathbf{Search-MCSP}}$) requiring circuits of size $\frac{2^n}{2n}$.*

Proof. Recall the two reductions in Lemma 2 and Theorem 7. In order to find an empty pigeonhole of the input circuit C given a solution to ϵ -**HARD**, we only need to use the $\mathcal{C}$ -inverter on C itself. In the case of a reduction from **HARD TRUTH TABLE** to ϵ -**HARD**, the circuit of interest C maps circuits of size at most $\frac{N}{2 \log N}$ to their N -bit truth tables, and so an oracle for **Search-MCSP** would suffice to invert C . $\square$

It should be noted that a related result was proven in [25], showing that this type of hardness amplification is possible in $\mathbf{E}$ assuming $\mathbf{MCSP} \in \mathbf{P}$. However, their proof does not translate directly to an unconditional result in the oracle setting. Due to their use of the Impagliazzo-Wigderson generator [20], directly applying their proof in the oracle setting using the relativized generator of [27] would instead show that if $\mathbf{E}^{\mathbf{MCSP}}$ requires $2^{\Omega(n)}$ -sized *nondeterministic* circuits, then $\mathbf{E}^{\mathbf{MCSP}}$ requires $\frac{2^n}{2n}$ -sized *standard* circuits, which is a weaker statement than what is shown above (modulo the search/decision distinction between **SEARCH-MCSP** and **MCSP**). Another result of a similar flavor was also proven in [17], where they establish that, assuming the (unproven) $\mathbf{NP}$ -completeness

of MCSP, $2^{n^{\Omega(1)}}$ lower bounds for $\mathbf{NP} \cap \mathbf{coNP}$ imply $2^{\Omega(n)}$ lower bounds for $\mathbf{E}^{\mathbf{NP}}$. This type of amplification is incomparable to the amplification demonstrated in Corollaries 2 and 3.

In [5], Buresh-Oppenheim and Santhanam define a notion of “hardness extraction” that is highly relevant to the results in this section. Informally, a hardness extractor is a procedure which takes a truth table of length N and circuit complexity s , and produces a truth table with nearly maximum circuit complexity relative to its size, whose length is as close to s as possible. The proof of Corollary 4 can in fact be viewed as a construction of a near-optimal hardness extractor using an sMCSP oracle. In particular our procedure is able to extract approximately the square root of the input’s hardness:

Theorem 11. *There is a polynomial time algorithm using an SEARCH-MCSP oracle which, given a truth table x of length M and circuit complexity s , outputs a truth table y of length $N = \Omega(\sqrt{\frac{s}{\log M}})$ and circuit complexity $\Omega(\frac{N}{\log N})$.*

Proof. The proof follows from a more careful analysis of Corollary 4; we give a sketch here. Adapting the proof of Theorem 1, for any N we can efficiently construct a circuit C with N outputs and $\lfloor \frac{N}{2} \rfloor$ inputs, such that every N -bit string outside its range requires circuits of size $\delta(\frac{N}{\log N})$ for some fixed $\delta > 0$. In particular, it is clear from the proof of Theorem 1 that such a C can be constructed of circuit size $O(N^2)$. Now, for any choice of N, M let $k = k(N, M)$ to be the minimum integer such that $2^k \frac{N}{2} \geq M$. Following the argument in the proof of Theorem 7, we can construct a map $C^* : \{0, 1\}^{\lfloor \frac{N}{2} \rfloor} \rightarrow \{0, 1\}^{2^k \lfloor \frac{N}{2} \rfloor}$ such that any element of its range has circuit complexity $O(|C|k) = O(N^2 k)$, and such that given a string outside the range of C^* , we can find a string outside the range of C using 2^k calls to an SEARCH-MCSP oracle. Now, we set N as a function of M by $N = \epsilon \sqrt{\frac{s}{\log M}}$ for ϵ sufficiently small. We get a value of $k = k(N, M) \leq \log M$.

This in turn means that a circuit of size $O(N^2 k) = O(\epsilon^2 s)$ whose truth table is $x0^{\lfloor \frac{N}{2} \rfloor 2^k - M}$ would contradict the fact that x has hardness at least s (for sufficiently small choice of ϵ). Thus, $x0^{\lfloor \frac{N}{2} \rfloor 2^k - M}$ must lie outside the range of C^* , so using x and our SEARCH-MCSP oracle we can find an N -bit string outside the range of C . This process requires at most $2^k \leq M$ calls to our SEARCH-MCSP oracle, each of input size at most $N < M$, so overall this takes polynomial time with access to an SEARCH-MCSP oracle. $\square$

A natural goal would be to improve this procedure to extract $(\frac{s}{\log M})^{\frac{1}{2} + \epsilon}$ or ideally $\Omega(\frac{s}{\log M})$ bits of hardness. The only obstacle here is improving the $O(N^2)$ upper bound on the circuit complexity of C , the circuit which takes descriptions of $\log N$ -input circuits of size $\approx N$ and outputs their truth tables. However, Williams observes in [45] (see footnote 7 of that paper) that an $N^{2-\epsilon}$ upper bound on $\text{size}(C)$ would imply a (nonuniform) breakthrough for 3SUM, so improving this extractor in its current form appears difficult.

5 Direct P Reductions to HARD TRUTH TABLE

Ideally we could extend the completeness result in Theorem 9 to work with polynomial time reductions, as opposed to $\mathbf{P}^{\mathbf{NP}}$ reductions. However, the $\mathbf{NP}$ oracle seems highly necessary for the proof techniques used above. Indeed, subsequent to the initial publication of this work, [19] gave strong cryptographic evidence that the completeness result in Theorem 9 requires an $\mathbf{NP}$ oracle, a result which we will discuss further in Section 6. Despite this obstacle, we show that there is a natural set of problems in $\mathbf{APEPP}$ which can be reduced to the problem of finding truth tables of hard functions via $\mathbf{P}$ reductions.

A simple way to phrase the following results is that any truth table with sufficiently large circuit complexity will necessarily satisfy a variety of other pseudorandom properties for which no explicit constructions are known, including: rigidity over $\mathbb{F}_2$, high space-bounded communication complexity, and high bit-probe complexity. To show this, we demonstrate that the failure of a string x to possess any of these properties implies a smaller than worst case circuit for x .

To give the tightest reductions possible, we will introduce one new parameterized version of the hard truth table construction problem:

Definition 19. δ -QUITE HARD is the following problem: given 1^N , output an N -bit truth table with hardness $\frac{\delta N}{\log N}$

This problem is total for sufficiently small δ . We recall also the definition of ϵ -HARD, where we must construct a truth table of hardness N^ϵ .

5.1 Rigidity

We begin with the case of rigidity. We recall the definition of the search problem $(\epsilon, 2)$ -RIGID, wherein given 1^N we are asked to produce an $N \times N$ matrix over $\mathbb{F}_2$ which requires changing $\geq \epsilon N^2$ entries to reduce its rank to ϵN . Any matrix achieving $(\epsilon, 2)$ rigidity for an absolute constant $\epsilon > 0$ (or even $\epsilon \gg \frac{1}{\log \log N}$) would be sufficient to carry out Valiant's program [44].

Theorem 12. For any sufficiently small $\delta > 0$, there exists some $\epsilon > 0$ such that ϵ -RATHER RIGID reduces in polynomial time to δ -QUITE HARD.

Proof. To prove this, it suffices to show that for any $N \times N$ matrix M which is not $(\epsilon N, \epsilon N^2)$ -rigid, we can construct a Boolean circuit with $f(\epsilon)O(\frac{N^2}{\log N})$ gates which decides the value of $M_{i,j}$ given the $2\lceil \log N \rceil$ -bit input (i, j) , for some function f which approaches zero as ϵ approaches zero. This then implies that for any fixed δ , an N^2 -bit truth table requiring circuits of size $\frac{\delta N^2}{\log N}$ must be $(\epsilon N, \epsilon N^2)$ -rigid for some $\epsilon > 0$ which is a function only of δ (and otherwise determined by f and the constants hidden in the $O(\cdot)$ term).

Say M is not $(\epsilon N, \epsilon N^2)$ -rigid. So there exists an $N \times \epsilon N$ matrix L , an $\epsilon N \times N$ matrix R , and an $N \times N$ matrix S with at most ϵN^2 nonzero entries, such that $LR \oplus S = M$. We will construct a circuit allowing us to efficiently index M which uses these matrices L, R, S as advice. To encode L and R , we can utilize the well-known theorem of Shannon that any truth table of length N can be computed by a circuit of size $O(\frac{N}{\log N})$ [40]. Thus, L can be specified as a list of ϵN circuits, each of size $O(\frac{N}{\log N})$, where the j^{th} circuit L_j represents the j^{th} column, and $L_j(i)$ computes $L_{i,j}$. The same can then be done for R (indexing columns instead). To encode S , we employ a refinement of Shannon's result due to Lupanov [16], which tells us that for sufficiently large N , any truth table of length N with at most ϵN nonzero entries can be computed by circuit of size

$$\frac{\log \binom{N}{\epsilon N}}{\log \log \binom{N}{\epsilon N}} + o\left(\frac{N}{\log N}\right) \leq H(\epsilon)O\left(\frac{N}{\log N}\right)$$

Where H denotes the binary entropy function. Thus S can be computed by a circuit of size $H(\epsilon)O(\frac{N^2}{\log N})$.

It remains to show that the additional circuitry we need to compute $M_{i,j}$ given i, j and the encodings of L, R, S does not increase things too much. By definition, we have that:

$$M_{i,j} = \langle \text{row}_i(L), \text{col}_j(R) \rangle \oplus S_{i,j}$$

where the dot product is taken over $\mathbb{F}_2$. Now consider the circuit diagram shown in Figure 3:

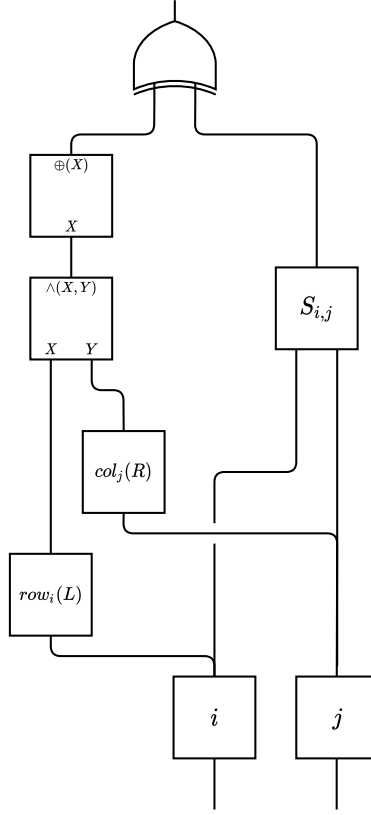


Figure 3: A small circuit for a non-rigid truth table

The subcircuit computing $\text{row}_i(L)$ is defined as follows: as described above, L represents a $N \times \epsilon N$ matrix, specified as a list of ϵN circuits of size $O(\frac{N}{\log N})$, each computing a column of L . $\text{row}_i(L)$ feeds i into each of these circuits in parallel to get an ϵN -bit string giving value of the i^{th} row of L . $\text{col}_j(R)$ is defined analogously. The circuit $S_{i,j}$ simply computes the $(i,j)^{\text{th}}$ entry of S given i,j . Finally, the subcircuit $\oplus(X)$ computes the parity of its input string, the subcircuit $\wedge(X,Y)$ computes the bit-wise AND of two equal length strings, and the terminal gate computes the two-bit parity function. Given the previous equation relating the $(i,j)^{\text{th}}$ index of M to the $(i,j)^{\text{th}}$ rows/columns/indices of L, R, S , it is straightforward to see that this circuit performs the necessary calculation.

It is clear that the $\oplus(X)$ and $\wedge(X,Y)$ can be implemented with a number of gates linear in their input size, which in this case is ϵN . From the analysis above, each of $\text{row}_i(L)$ and $\text{col}_j(R)$ can be implemented using $\epsilon O(\frac{N^2}{\log N})$ gates, and $S_{i,j}$ can be implemented using $H(\epsilon)O(\frac{N^2}{\log N})$ gates. So overall, this circuit has size $(H(\epsilon) + \epsilon)O(\frac{N^2}{\log N})$. Since $H(\epsilon) + \epsilon$ approaches zero for decreasing ϵ , this implies that for any fixed $\delta > 0$, an N^2 -bit truth table requiring circuits of size $\frac{\delta N^2}{\log N}$ must be $(\epsilon N, \epsilon N^2)$ rigid for some $\epsilon > 0$ which is a function only of δ (and which is otherwise determined by the constants hidden in the $O(\cdot)$ expressions above). $\square$

We thus conclude that if $\mathbf{E}$ contains a language of circuit complexity $\Omega(\frac{2^n}{n})$, then there is a polynomial time construction of $(\Omega(n), \Omega(n^2))$ -rigid matrices over $\mathbb{F}_2$.

5.2 Space-Bounded Communication Complexity

The class $\mathbf{PSPACE}^{\text{CC}}$ was defined originally in [2] as a generalization of the class $\mathbf{PH}^{\text{CC}}$ to an unbounded alternation of quantifiers. We will not give this original definition, but rather a simplification due to [42].

Definition 20. Let $f : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$. We say f has a space- s protocol if there is a deterministic protocol deciding f of the following form. Alice receives $x \in \{0,1\}^n$, and Bob receives $y \in \{0,1\}^n$. There is an s -bit shared memory, and Alice and Bob alternate turns writing to this memory. On a player's turn, they can modify the contents of the s -bit shared memory as a function only of its previous contents and their private input x or y , or decide to halt and output some $z \in \{0,1\}$ (this decision is also a function only of their input and the shared memory's previous state). This protocol is valid if $f(xy) = z$ for all x, y .

By a result of Song [42], we have that if $f \in \mathbf{PSPACE}^{\text{CC}}$ then f has a $\text{poly}(\log n)$ space protocol (Song uses a slightly different model where Alice and Bob have private s -bit tapes, but our model is at least as strong up to a doubling in space since sharing the tape only increases their ability to communicate). Due to a basic counting argument, most functions f require $\Omega(n)$ space, and any such function must lie outside of $\mathbf{PSPACE}^{\text{CC}}$. However, it has been a long standing open problem to give an explicit construction a communication matrix outside of even $\mathbf{PH}^{\text{CC}}$ [42]. We thus define the following search problem:

Definition 21. δ -SPACE is the following search problem. Given 1^N , where $N = 2^n$, output a communication matrix $\{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$ such that f requires space- δn communication protocols.

Theorem 13. For any $\delta < \epsilon < \frac{1}{2}$, δ -SPACE reduces in polynomial time to $(\frac{1}{2} + \epsilon)$ -HARD.

Proof. The proof follows the exact same strategy as above, constructing a smaller-than-worst-case circuit for any matrix with a δn -space protocol. We will omit the detailed construction of our circuit since it is very similar to that of the above proof, but give here a clear sketch of its essential structure. Let $f : \{0,1\}^n \times \{0,1\}^n \rightarrow \{0,1\}$, and let $N = 2^n$. We can represent a space s protocol for f as $2s + 4$ different $N \times 2^s$ binary matrices, with each of the first $2s$ matrices telling us how one of the two players will modify a certain cell of the shared memory as a function of its previous state and their input, and the last four matrices determining whether a certain player will halt or continue and the value they will output if they halt, again as a function of their input and the shared memory contents. For $s = \epsilon \log N$ (with $\epsilon < 1$), we can then take these matrices as advice to our circuit, for a total of $O(\epsilon N^{1+\epsilon} \log N)$ bits of advice. We can then add circuitry to simulate $2^{s+1} = 2N^\epsilon$ steps of this protocol on a given input $x, y \in \{0,1\}^n \times \{0,1\}^n$, indexing into the advice matrices in order to determine what to do next using the same indexing constructions we had for the proof of Theorem 12. Any matrix solvable by a space s protocol will be solved by a protocol that halts after 2^{s+1} steps (otherwise it will loop forever), so simulating for 2^{s+1} steps suffices. Since each of the indexing operations can be implemented using a number of gates linear in $\epsilon N^{1+\epsilon} \log N$ using the constructions from the proof of Theorem 12, our overall circuit for f will have size $O(2N^\epsilon \epsilon N^{1+\epsilon} \log N) = O(N^{1+2\epsilon} \log N)$. Thus, for any $\epsilon < \frac{1}{2}$, a truth table of length N^2 requiring circuits of size $N^{1+2\epsilon}$ will require $\delta \log N$ -space protocols for any $\delta < \epsilon$. $\square$

5.3 Bit Probe Lower Bounds

In [11], Elias and Flower defined a broad model for studying the space/query complexity of data structure problems, known as the “bit-probe model.”

Definition 22. Given two sets D, Q , a function $f : D \times Q \rightarrow \{0, 1\}$ and an integer b , the bit-probe complexity of f for space b , denoted $BC_b(f)$, is the minimum over all encodings $G : D \rightarrow \{0, 1\}^b$ of the number of bits of $G(x)$ that need to be probed in order to determine $f(x, y)$ for the worst case $x \in D, y \in Q$, given access to y .

In this general definition of a data structure problem, we think of D as the set of all possible pieces of “data” we might wish to encode in our data structure, b as the number of bits we can use to encode a piece of data, Q as the set of queries we wish to answer about an encoded piece of data, and f as telling us the correct answers to all data/query pairs. $BC_b(f)$ then tells us the minimum number of probes required by any space- b data structure in order to answer every query correctly for every possible piece of data.

This model was investigated further by Miltersen [30], who showed, using a simple counting argument, that most problems require an infeasible amount of space/probes in this model, but pointed out that no explicit data structure problem is known to be infeasible in this sense. We thus define the following explicit construction problem:

Definition 23. δ -PROBE is the following search problem: given 1^N where $N = 2^n$ output a truth table $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$ such that $BC_{2^{\delta n}}(f) > \delta n$.

Theorem 14. For any $\delta < 2\epsilon < 1$, δ -PROBE reduces in polynomial time to $(\frac{1}{2} + \epsilon)$ -HARD.

Proof. Say f is a function $f : D \times Q \rightarrow \{0, 1\}$ such that $BC_b(f) \leq k$. So in particular there is an encoding $G : D \rightarrow \{0, 1\}^b$ such for any $x \in D, y \in Q$, given access to y we can determine $f(x, y)$ using at most k probes to $G(x)$. Let $H : Q \rightarrow [b]^k$ be the function which, given y , tells us which positions of $G(x)$ to query. Finally, let $\phi : \{0, 1\}^k \times Q \rightarrow \{0, 1\}$ be the function which determines $f(x, y)$ given the results of the probes and the value of y . We will now give a compact representation for f , and then use it to construct a circuit computing f .

Let R be a $|D| \times b$ binary matrix whose rows are indexed by elements of D , where $R_{i,j}$ gives the value of the j^{th} bit of $G(i)$. Let S be a $|Q| \times k \lceil \log b \rceil$ matrix whose rows are indexed by elements of Q , such that the i^{th} row of S is $H(i) \in [b]^k$. Finally, let Z be a $2^k \times |Q|$ matrix such that $Z_{i,j} = \phi(i, j)$.

Now, given $x \in D, y \in Q$, we can use S, R, Z to determine $f(x, y)$ as follows. First find the y^{th} row of S to get $H(y)$, which is a list of k indices in $[b]$. Next, find the x^{th} row of R , which is the b -bit string $G(x)$. Then, probe the indices specified by $H(y)$ to get some k -bit string w , and finally output $Z_{w,y}$, which is precisely $\phi(w, y) = f(x, y)$.

As in the previous proofs, it can easily be shown that these indexing operations can be accomplished with circuits of size linear in S, R, Z . Therefore, f has a circuit of size $O(|S| + |R| + |Z|) = O(|Q|k \log b + |D|b + 2^k|Q|)$. So for any function $D \times Q \rightarrow \{0, 1\}^n$ requiring circuits of size $\omega(|Q|k \log b + |D|b + 2^k|Q|)$, we must have $BC_b(f) > k$.

In particular, if we take $D = Q = \{0, 1\}^n$, and for any fixed $\epsilon < 1$ we take $b = 2^{\epsilon n}$, $k = \epsilon n$, we get that for any function $f : \{0, 1\}^{2n} \rightarrow \{0, 1\}$ requiring circuits of size $\omega(2^{(1+\epsilon)n})$, it must be the case that $BC_{2^{\epsilon n}}(f) > \epsilon n$, since

$$|Q|k \log b + |D|b + 2^k|Q| = 2^n \epsilon^2 n^2 + 2^n 2^{\epsilon n} + 2^{\epsilon n} 2^n = O(2^{(1+\epsilon)n})$$

So if we take $\delta < \epsilon$, any truth table of length N^2 with hardness $N^{1+\epsilon}$, or in other words any solution to $(\frac{1}{2} + \frac{\epsilon}{2})$ -HARD on input 1^{N^2} , must satisfy $BC_{2^{\delta n}}(f) > \delta n$. $\square$

5.4 Some Concluding Thoughts

As noted in Section 3, Theorem 13 and Theorem 14 immediately imply that the problems δ -SPACE and δ -PROBE lie in **APEPP**, since we can construct truth tables with hardness $\frac{N}{2 \log N}$ in **APEPP** by Theorem 1. Although we phrase these results as reductions, they can also be interpreted as giving conditional polynomial time constructions of various objects, under different circuit lower bound assumptions for the class **E**. In particular, Theorems 13 and 14 establish that if **E** contains a language of circuit complexity $2^{(\frac{1}{2}+\epsilon)n}$ for some $\epsilon > 0$, then polynomial time constructions of hard problems in the bit-probe and space-bounded communication models follow. Theorem 12 establishes that if **E** contains a language of circuit complexity $\Omega(\frac{2^n}{n})$, then polynomial time constructions of $(\Omega(n), \Omega(n^2))$ -rigid matrices over $\mathbb{F}_2$ follow; such matrices would be sufficient to carry out Valiant's lower bound program.

It would be interesting as well to find some natural explicit construction problem for which a reduction exists in the opposite direction, i.e. this problem is at least as hard as **HARD TRUTH TABLE** or perhaps ϵ -**HARD**. Aside from $K_U^{n^2}$ -**RANDOM** for which such a reduction is immediate, we do not know of any other examples. However, we observe the following dichotomy: any explicit construction problem either has a non-trivial algorithm, or is at least as difficult as constructing a somewhat hard truth table. More precisely:

Lemma 3. *Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be non-increasing, and let Π be any property (language) recognizable in complexity class **C**. Let Π -CONSTRUCTION be the search problem: given 1^n , output an n -bit string with property Π . If Π -CONSTRUCTION is total for sufficiently large n , then for sufficiently large n one of the following holds:*

1. *There is a **TIME**($2^{\tilde{O}(f(n))}$) algorithm using a **C**-oracle that solves Π -CONSTRUCTION on length n inputs.*
2. *There is a polynomial time reduction from the problem of constructing an n -bit truth table with circuit complexity $f(n)$ to Π -CONSTRUCTION on length n inputs.*

Proof. The proof is a straightforward application of the “easy witness” paradigm [24]. Let Π^n be the set of n -bit strings with property Π and let H_f^n be the set of n -bit strings with circuit complexity at most $f(n)$. If $\Pi^n \cap H_f^n = \emptyset$ then any solution to the explicit construction problem for Π is necessarily a string with circuit complexity exceeding $f(n)$, and hence a polynomial time reduction from truth table construction to Π -CONSTRUCTION trivially follows. On the other hand, if $\Pi^n \cap H_f^n \neq \emptyset$, we can search over H_f^n for a solution to Π -CONSTRUCTION and will be guaranteed to find a solution. Since $|H_f^n| = 2^{\tilde{O}(f(n))}$, we can then solve Π -CONSTRUCTION by iterating over H_f^n and using a **C**-oracle to test if each potential solution indeed holds property Π , in **TIME**($2^{\tilde{O}(f(n))}$). $\square$

Setting $f(n) = n^\epsilon$ and combining this with Theorem 7, we conclude:

Theorem 15. *Let EC be any explicit construction problem in **APEPP** (more formally, any search problem in $\mathbf{STF}\Sigma_2^P \cap \mathbf{APEPP}$). Then one of the following holds:*

1. *EC is **APEPP**-complete under $\mathbf{P}^{\mathbf{NP}}$ reductions.*
2. *For every $\epsilon > 0$, EC can be solved in time 2^{n^ϵ} with an **NP** oracle, for infinitely many n .*

6 Open Problems and Subsequent Work

In the original publication of this work, the primary open problem highlighted was to either show that ϵ -HARD is complete for **APEPP** under polynomial time reductions (rather than $\mathbf{P}^{\mathbf{NP}}$ reductions), or give complexity-theoretic evidence against this possibility. As observed in the original manuscript, a sufficient condition for the impossibility of a polynomial time reduction from EMPTY to ϵ -HARD is that EMPTY is not solvable in **FP/poly**; in this case no $\mathbf{TF}\Sigma_2^{\mathbf{P}}$ search problem with *sparse input* (i.e., no explicit construction problem) can be hard for **APEPP** under polynomial time reductions, since the solution to a sparse search problem can be hard-coded by polynomial size advice. In subsequent work of Ilango, Li and Williams [19], it was shown that EMPTY is not solvable in **FP/poly** under two hardness assumptions: the existence of Indistinguishability Obfuscation and $\mathbf{NP} \neq \mathbf{coNP/poly}$. Indistinguishability Obfuscation is a very powerful cryptographic primitive, which in turn was recently shown to exist under relatively standard concrete cryptographic assumptions based on random linear codes, bilinear groups, and pseudorandom generators with large stretch in $\mathbf{NC}^0$ [21, 22]. It remains an open problem whether the hardness of EMPTY can be based on a more standard hardness assumption such as the infinitude of the polynomial hierarchy.

7 Acknowledgements

The author would like to thank Christos Papadimitriou for his guidance and for many inspiring discussions throughout the completion of this work, and Mihalis Yannakakis for his comments on an early draft of this manuscript. The author would also like to thank the anonymous referees for suggesting various improvements to this paper, in particular the addition of Corollary 3, the connection to hardness extractors and the GGM generator, and the simplification of Lemma 1.

References

- [1] J. ALMAN AND L. CHEN, *Efficient construction of rigid matrices using an NP oracle*, in 2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS), 2019, pp. 1034–1055.
- [2] L. BABAI, P. FRANKL, AND J. SIMON, *Complexity classes in communication complexity theory*, in 27th Annual Symposium on Foundations of Computer Science (sfcs 1986), 1986, pp. 337–347.
- [3] B. BARAK, A. RAO, R. SHALTIEL, AND A. WIGDERSON, *2-source dispersers for sub-polynomial entropy and ramsey graphs beating the frankl-wilson construction*, in Proceedings of the Thirty-Eighth Annual ACM Symposium on Theory of Computing, STOC '06, New York, NY, USA, 2006, Association for Computing Machinery, p. 671–680.
- [4] A. BHANGALE, P. HARSHA, O. PARADISE, AND A. TAL, *Rigid matrices from rectangular PCPs*, 2020.
- [5] J. BURESH-OPPENHEIM AND R. SANTHANAM, *Making hard problems harder*, 21st Annual IEEE Conference on Computational Complexity (CCC'06), (2006), pp. 15 pp.–87.
- [6] E. CHATTOPADHYAY, *Guest column: A recipe for constructing two-source extractors*, SIGACT News, 51 (2020), p. 38–57.
- [7] L. CHEN, S. HIRAHARA, AND H. REN, *Symmetric exponential time requires near-maximum circuit size*, in 56th Annual Symposium on Theory of Computing, 2024.
- [8] L. CHEN, Z. LU, I. C. OLIVEIRA, H. REN, AND R. SANTHANAM, *Polynomial-time pseudodeterministic construction of primes*, in 2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS), 2023, pp. 1261–1270.
- [9] B. CHOR AND O. GOLDBREICH, *Unbiased bits from sources of weak randomness and probabilistic communication complexity*, in 26th Annual Symposium on Foundations of Computer Science (sfcs 1985), 1985, pp. 429–442.

- [10] Z. DVIR, A. GOLOVNEV, AND O. WEINSTEIN, *Static data structure lower bounds imply rigidity*, in Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, New York, NY, USA, 2019, Association for Computing Machinery, p. 967–978.
- [11] P. ELIAS AND R. A. FLOWER, *The complexity of some simple retrieval problems*, J. ACM, 22 (1975), p. 367–379.
- [12] P. ERDÖS, *Some remarks on the theory of graphs*, Bulletin of the American Mathematical Society, 53 (1947), pp. 292–294.
- [13] J. FRIEDMAN, *A note on matrix rigidity*, Combinatorica, 13 (1993), pp. 235–239.
- [14] E. GAT AND S. GOLDWASSER, *Probabilistic search algorithms with unique answers and their cryptographic applications*, Electronic Colloquium on Computational Complexity (ECCC), 18 (2011), p. 136.
- [15] O. GOLDBREICH, S. GOLDWASSER, AND S. MICALI, *How to construct random functions*, J. ACM, 33 (1986), p. 792–807.
- [16] A. GOLOVNEV, R. ILANGO, R. IMPAGLIAZZO, V. KABANETS, A. KOLOKOLOVA, AND A. TAL, *$Ac0[p]$ lower bounds against mcs_p via the coin problem*, Electron. Colloquium Comput. Complex., 26 (2019), p. 18.
- [17] J. M. HITCHCOCK AND A. PAVAN, *On the np -completeness of the minimum circuit size problem*, in FSTTCS, 2015.
- [18] R. ILANGO, *Approaching MCSP from Above and Below: Hardness for a Conditional Variant and $AC^0[p]$* , in 11th Innovations in Theoretical Computer Science Conference (ITCS 2020), T. Vidick, ed., vol. 151 of Leibniz International Proceedings in Informatics (LIPIcs), Dagstuhl, Germany, 2020, Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, pp. 34:1–34:26.
- [19] R. ILANGO, J. LI, AND R. R. WILLIAMS, *Indistinguishability obfuscation, range avoidance, and bounded arithmetic*, in Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, New York, NY, USA, 2023, Association for Computing Machinery, p. 1076–1089.
- [20] R. IMPAGLIAZZO AND A. WIGDERSON, *$P = BPP$ if E requires exponential circuits: Derandomizing the XOR lemma*, in Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing, STOC '97, New York, NY, USA, 1997, Association for Computing Machinery, p. 220–229.
- [21] A. JAIN, H. LIN, AND A. SAHAI, *Indistinguishability obfuscation from well-founded assumptions*, in Proceedings of the 53rd annual ACM SIGACT symposium on theory of computing, 2021, pp. 60–73.
- [22] ———, *Indistinguishability obfuscation from l_{pn} over f_p , d_{lin} , and $prgs$ in nc_0* , in Annual International Conference on the Theory and Applications of Cryptographic Techniques, Springer, 2022, pp. 670–699.
- [23] E. JEŘÁBEK, *Dual weak pigeonhole principle, boolean complexity, and derandomization*, Annals of Pure and Applied Logic, 129 (2004), pp. 1–37.
- [24] V. KABANETS, *Easiness assumptions and hardness tests: Trading time for zero error*, Journal of Computer and System Sciences, 63 (2001), pp. 236–252.
- [25] V. KABANETS AND J.-Y. CAI, *Circuit minimization problem*, in Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing, STOC '00, New York, NY, USA, 2000, Association for Computing Machinery, p. 73–79.
- [26] R. KLEINBERG, O. KORTEN, D. MITROPOLSKY, AND C. PAPADIMITRIOU, *Total Functions in the Polynomial Hierarchy*, in 12th Innovations in Theoretical Computer Science Conference (ITCS 2021), J. R. Lee, ed., vol. 185 of Leibniz International Proceedings in Informatics (LIPIcs), Dagstuhl, Germany, 2021, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, pp. 44:1–44:18.
- [27] A. R. KLIVANS AND D. VAN MELKEBEEK, *Graph nonisomorphism has subexponential size proofs unless the polynomial-time hierarchy collapses*, SIAM Journal on Computing, 31 (2002), pp. 1501–1526.
- [28] X. LI, *Two Source Extractors for Asymptotically Optimal Entropy, and (Many) More*, in 2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS), Los Alamitos, CA, USA, Nov. 2023, IEEE Computer Society, pp. 1271–1281.
- [29] P. MILTERSEN AND N. VINODCHANDRAN, *Derandomizing arthur-merlin games using hitting sets*, in 40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039), 1999, pp. 71–80.
- [30] P. B. MILTERSEN, *The bit probe complexity measure revisited*, in STACS 93, P. Enjalbert, A. Finkel, and K. W. Wagner, eds., Berlin, Heidelberg, 1993, Springer Berlin Heidelberg, pp. 662–671.
- [31] N. NISAN AND A. WIGDERSON, *Hardness vs randomness*, Journal of Computer and System Sciences, 49 (1994), pp. 149–167.
- [32] I. C. OLIVEIRA AND R. SANTHANAM, *Pseudodeterministic constructions in subexponential time*, in Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, New York, NY, USA, 2017, Association for Computing Machinery, p. 665–677.

- [33] C. H. PAPADIMITRIOU, *On the complexity of the parity argument and other inefficient proofs of existence*, Journal of Computer and System Sciences, 48 (1994), pp. 498 – 532.
- [34] J. PARIS, A. WILKIE, AND A. R. WOODS, *Provability of the pigeonhole principle and the existence of infinitely many primes*, J. Symb. Log., 53 (1988), pp. 1235–1244.
- [35] A. RAZBOROV, *Bounded-depth formula over $\{AND, XOR\}$ and some combinatorial problems (russian)*, Problems of Cybernetics. Complexity Theory and Applied Mathematical Logic, (1988), pp. 149–166.
- [36] A. A. RAZBOROV AND S. RUDICH, *Natural proofs*, Journal of Computer and System Sciences, 55 (1997), pp. 24–35.
- [37] H. REN, R. SANTHANAM, AND Z. WANG, *On the range avoidance problem for circuits*, in 2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS), 2022, pp. 640–650.
- [38] R. SANTHANAM, *The complexity of explicit constructions*, Theory of Computing Systems, 51 (2012), pp. 297–312. Copyright - Springer Science+Business Media, LLC 2012; Document feature - ; Equations; Last updated - 2020-11-18; CODEN - TCSYFI.
- [39] ———, *Pseudorandomness and the minimum circuit size problem*, in 11th Innovations in Theoretical Computer Science Conference, ITCS 2020, January 12-14, 2020, Seattle, Washington, USA, T. Vidick, ed., vol. 151 of LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, pp. 68:1–68:26.
- [40] C. E. SHANNON, *The synthesis of two-terminal switching circuits*, The Bell System Technical Journal, 28 (1949), pp. 59–98.
- [41] M. A. SHOKROLLAHI, D. SPIELMAN, AND V. STEMANN, *A remark on matrix rigidity*, Information Processing Letters, 64 (1997), pp. 283–285.
- [42] H. SONG, *Space-bounded Communication Complexity*, PhD thesis, Tsinghua University, 2014.
- [43] S. P. VADHAN, *Pseudorandomness*, vol. 7, Now Delft, 2012.
- [44] L. G. VALIANT, *Graph-theoretic arguments in low-level complexity*, in Mathematical Foundations of Computer Science 1977, J. Gruska, ed., Berlin, Heidelberg, 1977, Springer Berlin Heidelberg, pp. 162–176.
- [45] R. WILLIAMS, *Improving exhaustive search implies superpolynomial lower bounds*, in Proceedings of the Forty-Second ACM Symposium on Theory of Computing, STOC '10, New York, NY, USA, 2010, Association for Computing Machinery, p. 231–240.
- [46] A. C. YAO, *Theory and application of trapdoor functions*, in 23rd Annual Symposium on Foundations of Computer Science (sfcs 1982), 1982, pp. 80–91.